

Detecting Fraudulent Credit Card Transactions

An End-to-End Machine Learning Lifecycle Project

Allen Mendiola

Pillar 5 Capstone Assignment

Domain: Finance - Fraudulent Transaction Detection

21 September 2026

A complete machine learning lifecycle, from framing the business problem through to auditing the finished model for bias.

Contents

- Executive Summary
- Step 1 - Problem Understanding and Framing
- Step 2 - Data Collection and Understanding
- Step 3 - Preprocessing, Applied EDA and Feature Engineering
- Step 4 - Model Implementation
- Step 5 - Critical Thinking: Ethical AI and Bias Auditing
- Step 6 - Final Presentation and Communication
- Step 7 - GitHub Repository
- Conclusion and Recommendations
- Limitations and Future Work
- References
- Appendix A - How to Reproduce This Work

Executive Summary

This project builds and audits a machine learning system that spots fraudulent credit card transactions. It follows the full lifecycle: framing the problem, understanding the data, cleaning and engineering features, training and comparing seven models, explaining what the winning model has learned, and finally checking whether it treats different groups of customers fairly.

The dataset contains 1,048,575 card transactions made between 2019-01-01 and 2020-03-10. Only 6,006 of them are fraudulent - 0.573% of the total. That rarity is the central difficulty of the whole project, and it shapes every decision that follows.

Headline results

Best model: Random Forest

PR-AUC (the metric that matters here): 0.8605

ROC-AUC: 0.9954

Fraud caught at the chosen threshold: 1009 of 1145 (88.1%)

Estimated net benefit on the test period: \$554,950

A model that simply never predicted fraud would still be 99.45% accurate. That is why this report leads with PR-AUC and recall rather than accuracy - accuracy is actively misleading when the thing you are looking for is this rare.

What the fairness audit found

Gender and job were deliberately withheld from the model. Many people assume that this alone makes a model fair. The audit in Step 5 tests that assumption directly, and the results are reported honestly, including where the model falls short.

Gender - disparate impact ratio: 0.836 (passes the 80% rule)

Age - gap in fraud caught between best and worst served group: 4.5 percentage points

After mitigation: gap reduced to 0.1 points

Step 1 - Problem Understanding and Framing

1.1 The business problem

Card fraud is a direct financial loss. When a stolen card is used, the bank usually refunds the customer and absorbs the cost. The bank therefore wants to spot fraud while it is happening, so the payment can be stopped before the money is gone.

The difficulty is that blocking payments is not free. Every alert has to be checked by a person, and every wrongly blocked payment annoys an innocent customer who is standing at a till. A system that flags everything would catch all the fraud and destroy the business. The real task is therefore a balancing act, not simply 'find the fraud'.

1.2 Task type

Learning type: Supervised learning

Task: Binary classification - each transaction is either fraud (1) or genuine (0)

Target variable: is_fraud

Unit of analysis: One card transaction

Prediction timing: At the moment the payment is attempted

A secondary unsupervised task is also carried out in Step 4. Clustering and anomaly detection are used to check whether suspicious behaviour can be found without any fraud labels at all. This matters in practice because in a real bank the labels arrive weeks later, when customers report the fraud.

1.3 Success metrics

Two sets of measures are used, because the data science answer and the business answer are not the same thing.

Technical metrics

- PR-AUC (average precision) - the headline metric. It measures how well the model ranks fraud above genuine payments, and unlike ROC-AUC it does not flatter a model when the positive class is rare.
- Recall - of all the real fraud, what share did we catch? This is what the bank loses money on when it is low.
- Precision - of everything we flagged, what share was really fraud? This is what drives the review workload.
- ROC-AUC and F1 - reported for completeness and for comparison with other studies.

Accuracy is deliberately NOT used as a success metric. With a fraud rate of 0.573%, a model that predicts 'genuine' for everything scores 99.45% accuracy and catches zero fraud.

Business KPIs

- Value of fraud blocked, in dollars.
- Cost of investigating false alarms, assumed at \$5 per alert of analyst time.
- Net benefit = fraud blocked minus review cost. This single number is used in Step 4 to choose the alert threshold.
- Customer friction, measured as the number of genuine customers wrongly blocked.

1.4 Capstone linkage

The brief asks that the Module 1 output maps onto Capstone Steps 1 to 3. The mapping used here is:

Module 1 output	Capstone step	How it is addressed here
Problem statement	Step 1	Detect fraudulent card transactions in real time
Task type	Step 1	Supervised binary classification, plus unsupervised support
Target metric	Step 1	PR-AUC, with recall and net benefit as business measures
Data sourcing	Step 2	Public simulated transaction dataset, 1.05 million rows
Data dictionary	Step 2	All 22 variables documented in section 2.3
Preparation	Step 3	Cleaning, 13 engineered features, time-based split

Step 2 - Data Collection and Understanding

2.1 Source and licence

Dataset: Credit Card Transactions Fraud Detection (Sparkov simulation)

Hosted at: Hugging Face - santosh3110/credit_card_fraud_transactions

Access: Public download, no account or API key required

File used: credit_card_fraud_transactions.csv

The data is simulated rather than taken from a real bank. This is a deliberate choice and a necessary one: genuine card transaction data cannot be published because it contains real people's spending. The simulation was built to reproduce realistic spending patterns, and crucially it keeps the customer details - gender, date of birth, job, city size - that the fairness audit in Step 5 depends on.

The better known alternative, the ULB 'creditcard' dataset, was rejected for exactly this reason. Its features are anonymised into unnamed components V1 to V28, which would have made both the SHAP explanations and the bias audit impossible to carry out meaningfully.

2.2 Dataset overview

Property	Value
Rows (transactions)	1,048,575
Columns	22
Fraudulent transactions	6,006
Genuine transactions	1,042,569
Fraud rate	0.573%
Time covered	2019-01-01 to 2020-03-10
Missing values	0
Duplicate rows	0
Total value of fraud	\$3,186,624
Average fraudulent payment	\$531
Average genuine payment	\$68

The average fraudulent payment is \$531, roughly 7.8 times the size of the average genuine payment at \$68. This is the single strongest clue in the whole dataset.

2.3 Data dictionary

Every column in the raw file, what it means, and what values it can take.

Variable	Description	Type	Unit / format	Distinct values
trans_date_trans_time	Date and time the transaction happened	object	text date (M/D/YY H:MM)	476595
cc_num	Credit card number of the customer (identifier only)	float64	number	943
merchant	Name of the shop where the card was used	object	text	693
category	Type of purchase, for example grocery_pos or shopping_net	object	text (14 categories)	14
amt	Amount of the transaction	float64	US dollars	48602

Variable	Description	Type	Unit / format	Distinct values
first	Customer first name - PERSONAL DATA	object	text	348
last	Customer last name - PERSONAL DATA	object	text	479
gender	Gender of the cardholder - SENSITIVE ATTRIBUTE	object	M or F	2
street	Customer home street address - PERSONAL DATA	object	text	965
city	Customer home city	object	text	879
state	Customer home state	object	text (2-letter code)	51
zip	Customer home postcode	int64	number	952
lat	Latitude of the customer home	float64	degrees	950
long	Longitude of the customer home	float64	degrees	951
city_pop	Population of the customer city - used as a wealth/urban proxy	int64	number of people	865
job	Job title of the cardholder - SENSITIVE ATTRIBUTE (social status)	object	text	493
dob	Date of birth of the cardholder - used to compute age	object	text date (M/D/YY, 2-digit year)	950
trans_num	Unique reference code of the transaction	object	text (hash)	1048575
unix_time	Same transaction time but counted in seconds since 1970	int64	seconds	1030650
merch_lat	Latitude of the shop	float64	degrees	1016437
merch_long	Longitude of the shop	float64	degrees	1034825
is_fraud	TARGET: 1 means the transaction was fraud, 0 means it was genuine	int64	0 or 1	2

2.4 Data quality findings

Three issues were found while inspecting the data. All three are reported here rather than quietly fixed, because each one would change how much trust the results deserve.

Finding 1 - the file has been truncated

The file contains exactly 1,048,575 rows. That number is not a coincidence: it is precisely the maximum number of data rows Microsoft Excel can hold. The original Sparkov training file is known to contain 1,296,675 rows, so roughly 248,000 transactions have been lost, almost certainly because somebody opened the file in Excel and saved it again.

The practical effect is that the data stops on 2020-03-10 instead of running to mid-2020. Since the split between training and test data is by date, this simply means the test period is earlier than it would otherwise have been. It does not invalidate the results, but it is an honest limitation.

Finding 2 - dates of birth were being read in the wrong century

Birth dates are stored as text with a two-digit year, for example "1/19/62". Python's default rule treats any year below 69 as belonging to the 2000s, so 62 was read as 2062 rather than 1962. This affected 358,189 rows, 34.2% of the dataset, and produced customers who appeared to be born in the future with NEGATIVE ages.

This bug was caught because a clustering profile reported an average customer age of minus 41 years. Had it gone unnoticed, the age feature would have been meaningless and - far more seriously - the age fairness audit in Step 5 would have been auditing nonsense while appearing to work perfectly.

The fix, applied in script 03, is to subtract one hundred years from any birth date that falls after the transactions took place. Ages afterwards run from roughly 14 to 95 years, which is sensible.

Finding 3 - no missing values at all

There are zero missing values and zero duplicate transactions. Real banking data is never this clean. This is a reminder that the dataset is simulated, and that the missing-value handling demonstrated in Step 3 would be doing considerably more work on a real system.

Step 3 - Preprocessing, Applied EDA and Feature Engineering

3.1 Cleaning decisions

- Duplicate transactions were removed using the unique transaction reference. None were found, but the check remains in the code so it would catch them in future data.
- Both date columns are parsed with an explicit format rather than letting pandas guess, and the century bug described in section 2.4 is corrected.
- Rows with no target value are dropped, since nothing can be learned from them.
- Extreme transaction amounts are KEPT, not removed. This is the opposite of the usual advice. In fraud detection an unusually large payment is very often the fraud itself, so deleting outliers would delete the signal.

On that last point: 54,542 transactions (5.20% of the data) sit above the usual statistical cut-off of \$193. Removing them would have discarded a large share of the fraud the model is supposed to find.

3.2 Exploratory data analysis

The class imbalance

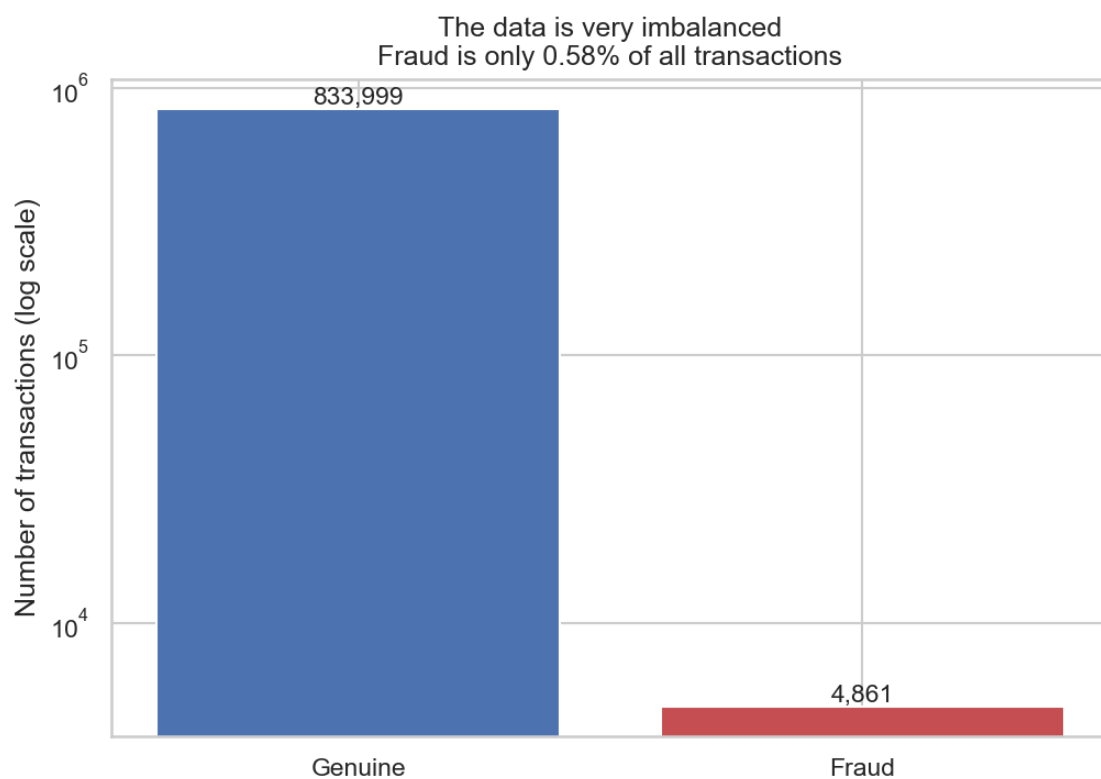


Figure 1: Fraud makes up only 0.580% of transactions. Note the logarithmic scale - on a normal scale the fraud bar would be invisible.

Transaction amount

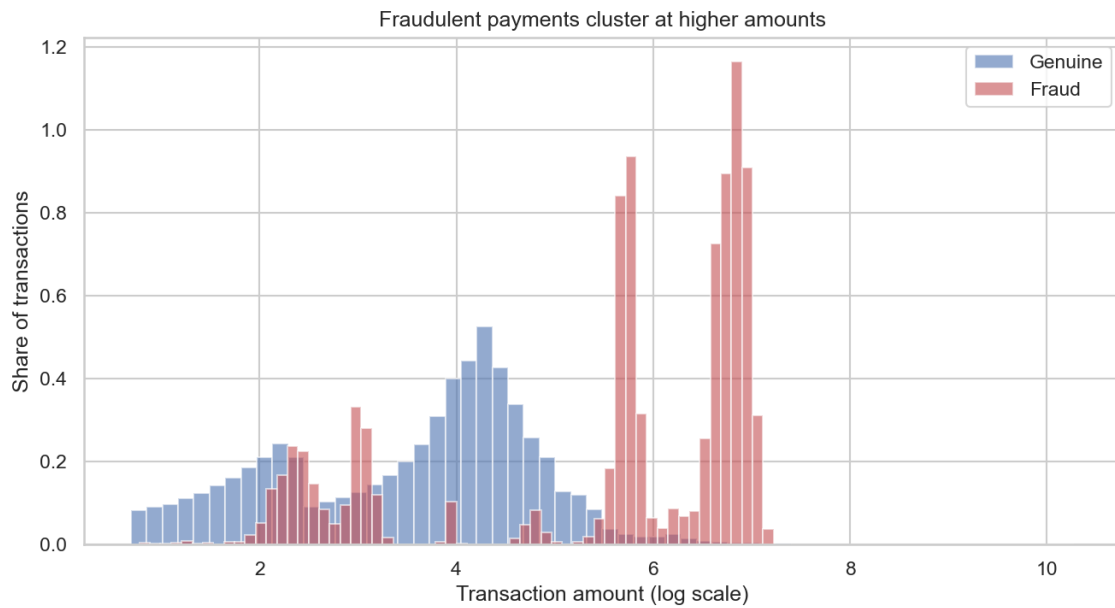


Figure 2: Fraudulent payments cluster at clearly higher amounts. Average fraud \$526 against \$68 for genuine payments.

Time of day

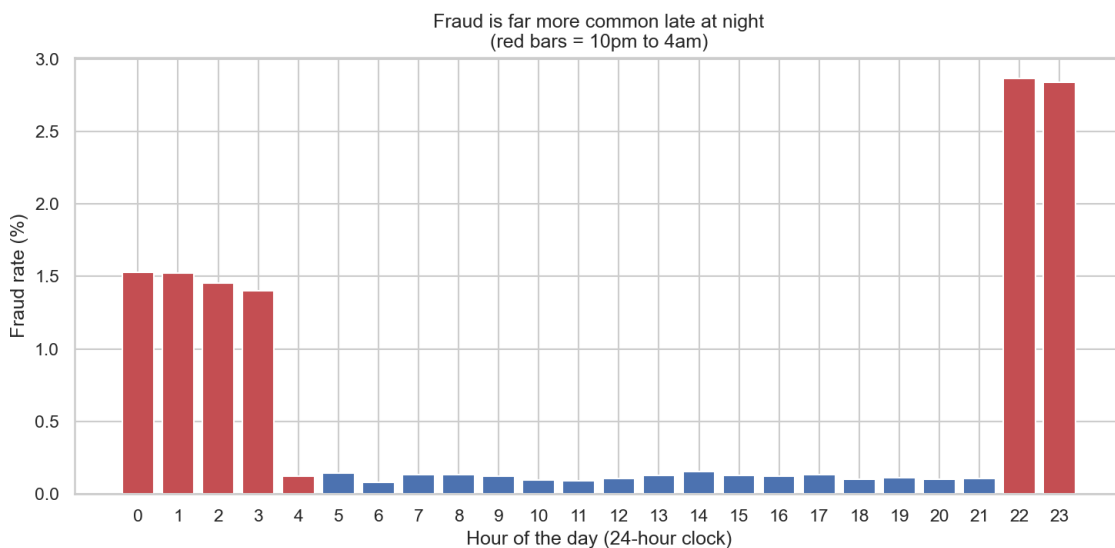


Figure 3: Fraud rate by hour. Between 10pm and 4am the rate is 1.842%, against 0.118% during the day - a difference of roughly 16 times.

This is the clearest single pattern in the data. The explanation is intuitive: a criminal using a stolen card prefers the hours when the real owner is asleep and cannot notice the payment or answer a verification call.

Purchase category

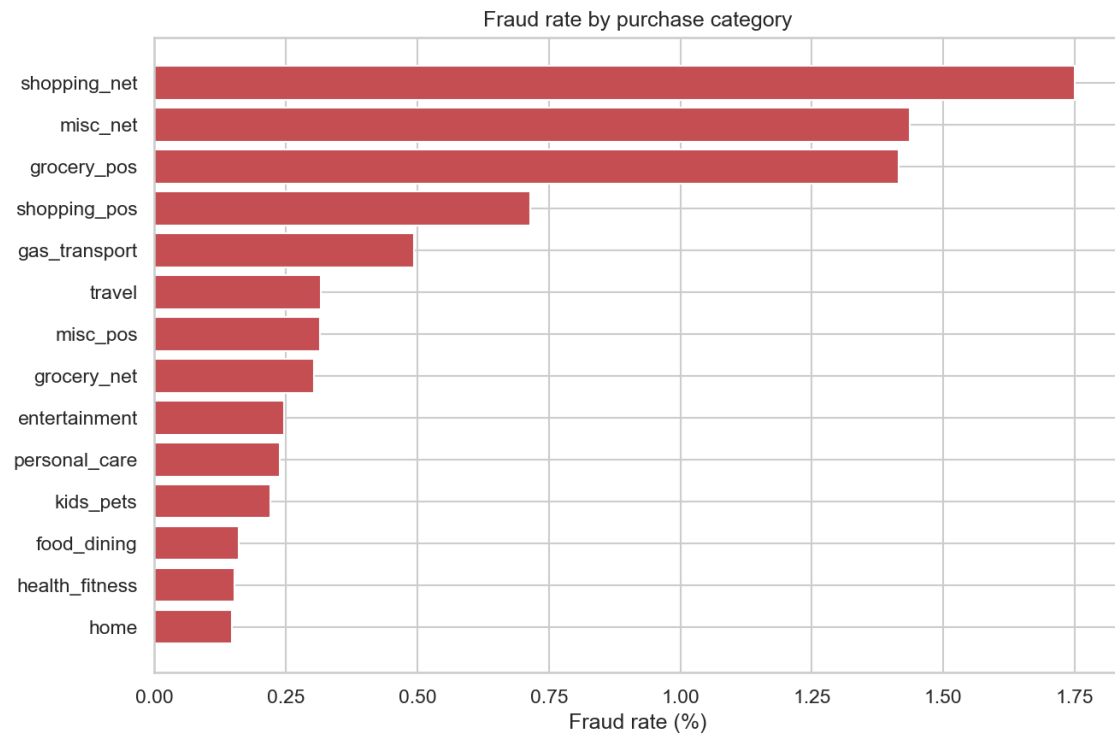


Figure 4: Fraud rate by category. 'shopping_net' is the riskiest at 1.75%, and the safest is 'home'.

Customer age

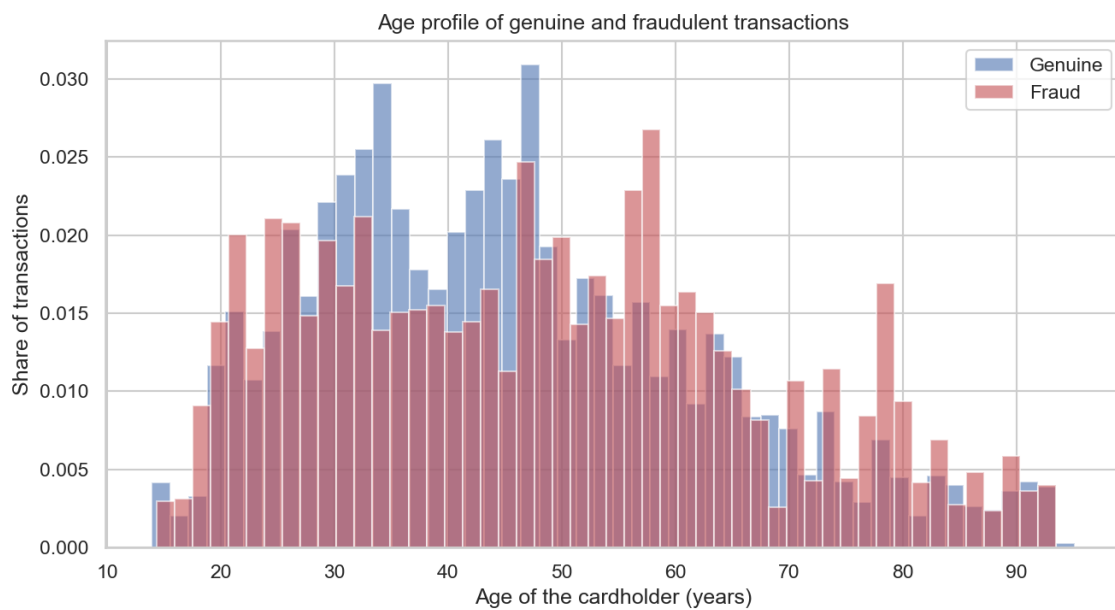


Figure 5: Age profile. Fraud victims average 48.3 years against 45.8 for genuine transactions.

How the features relate to each other

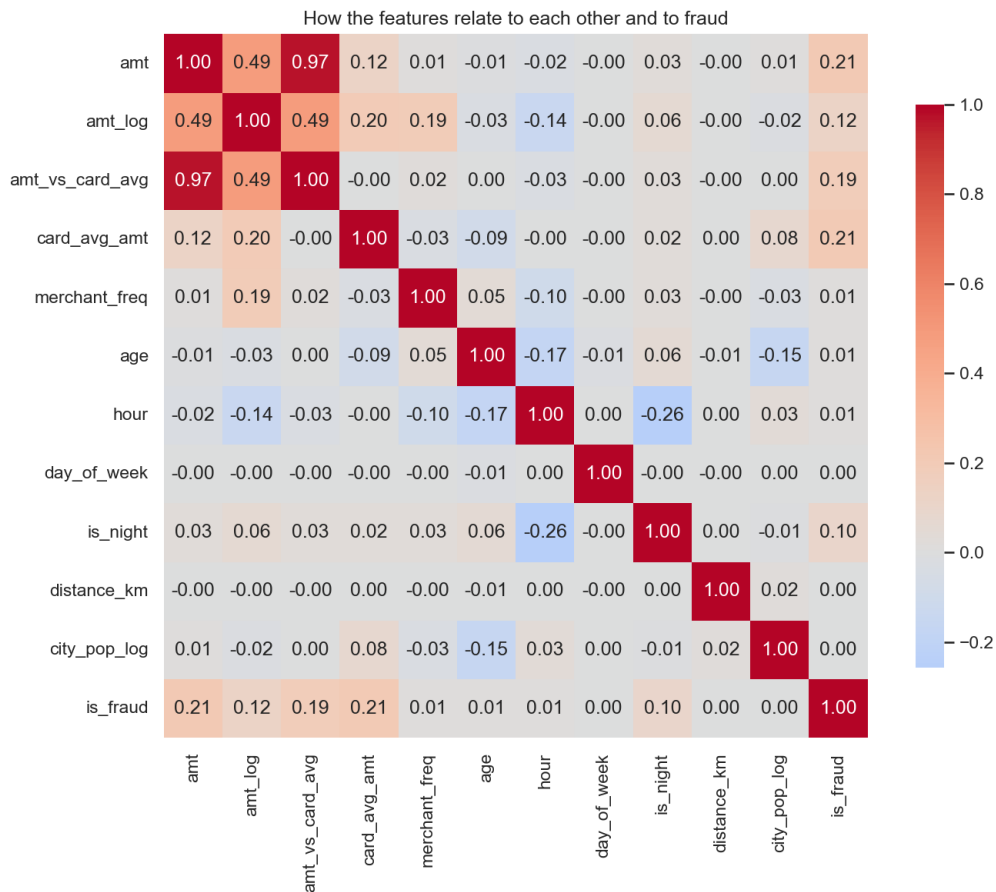


Figure 6: Correlation heatmap. The feature most strongly linked to fraud is 'amt'.

An early warning for the fairness audit

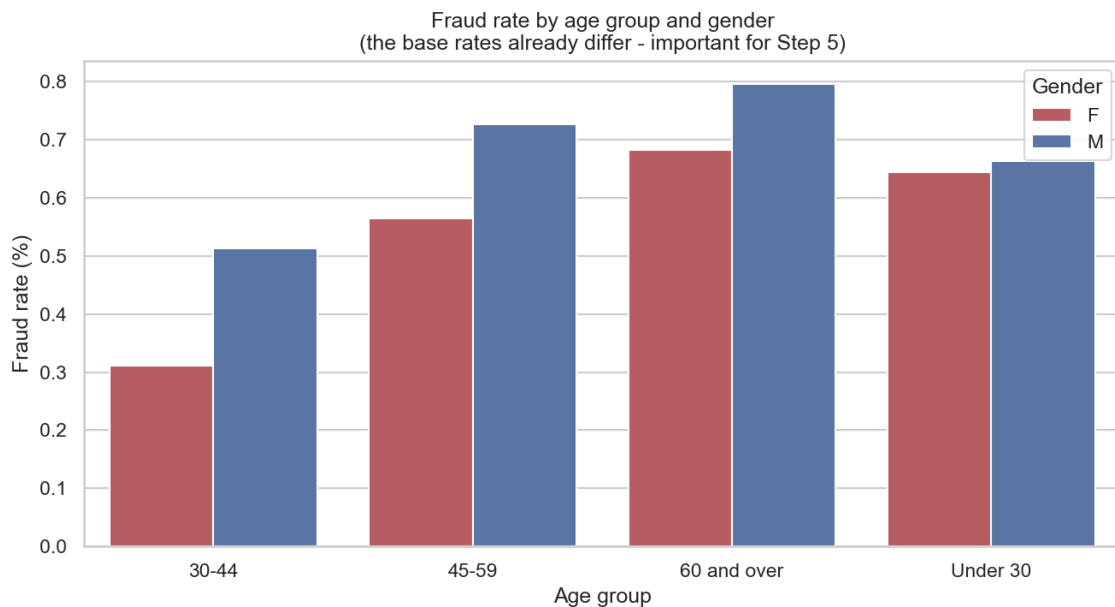


Figure 7: Fraud rates already differ by age group and gender before any model is trained. Those differences are in the world, not in the algorithm - but a model will learn them.

Distance and spending behaviour

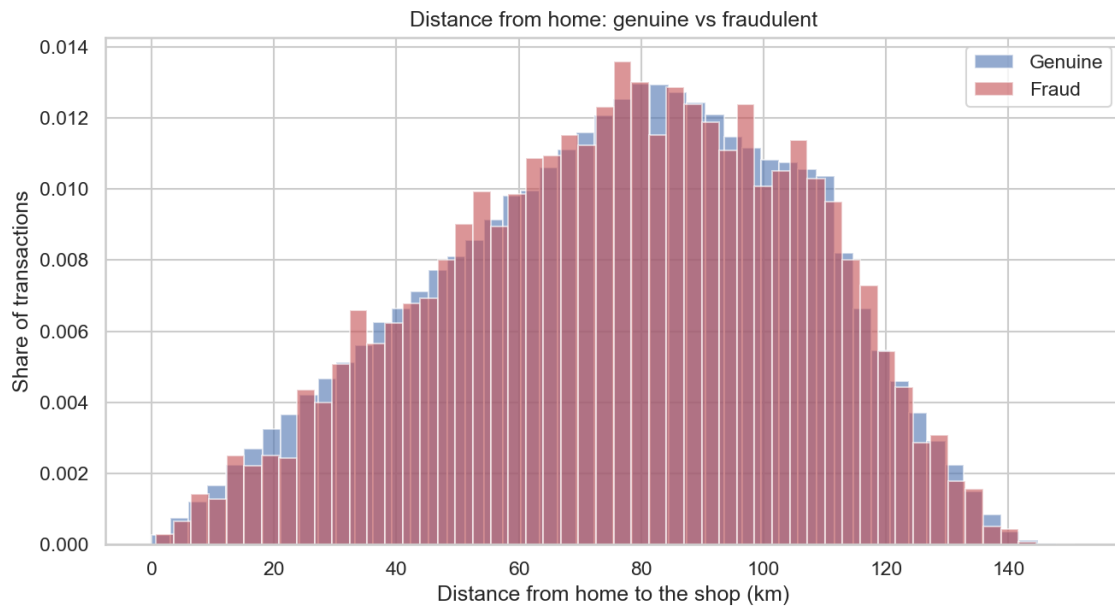


Figure 8: Distance between the customer's home and the shop.

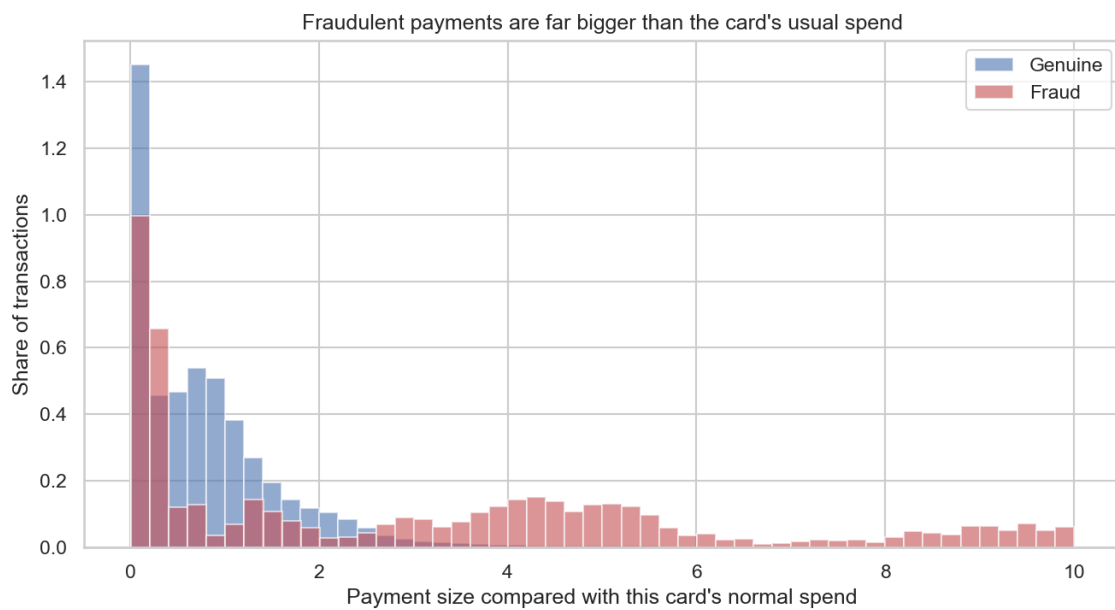


Figure 9: Payment size compared with what that particular card normally spends. Fraudulent payments are dramatically out of character.

3.3 Feature engineering

Thirteen features were engineered from the raw columns. Each one exists for a stated reason rather than because it was available.

Feature	Type	Why it is used
amt	Original	The payment amount. Fraudsters often test with small amounts then spend big.
amt_log	Engineered	Log of the amount. Squashes a very skewed range so models cope better.
amt_vs_card_avg	Engineered	Amount divided by this card's usual spend. Catches out-of-character payments.

Feature	Type	Why it is used
card_avg_amt	Engineered	The card's normal spending level, learned from training data only.
merchant_freq	Engineered	How often the shop appears. Unknown shops carry more risk.
age	Engineered	Age at the time of payment, worked out from date of birth.
hour	Engineered	Hour of the day. Fraud peaks at night.
day_of_week	Engineered	Day of the week, to capture weekly habits.
month	Engineered	Month, to capture seasonal effects.
is_night	Engineered	1 if between 10pm and 4am, when the real owner is usually asleep.
distance_km	Engineered	Distance from home to the shop, using the haversine formula.
city_pop	Original	Size of the customer's city, a rough proxy for urban or rural.
city_pop_log	Engineered	Log of city size, because the raw values are very skewed.
cat_*	Engineered	One yes/no column per purchase category (one-hot encoding).

Scaling, encoding and binning

Three standard preparation techniques are applied, each for a specific reason rather than out of habit.

Technique	What was done	Why
Scaling	StandardScaler on all 27 features	Logistic Regression, SVM and the neural network all measure distances between points. Without scaling, city population (in the millions) would completely drown out is_night (0 or 1). Tree models do not care, so both a scaled and an unscaled copy of the data are kept and each model is given the one it needs.
Encoding	One-hot for category (14 columns); frequency encoding for merchant	Models cannot read text. Category has only 14 values, so one-hot keeps each one separately interpretable for SHAP. Merchant has 693 values, which would add 693 columns, so it is replaced by how often each shop appears instead.
Binning	age into 4 bands; city_pop into 4 size classes	These bands are NOT model inputs. They exist so the fairness audit in Step 5 can compare groups. Auditing across continuous age would give one 'group' per customer and say nothing.

The scaler is fitted on the training rows only and then applied to the test rows. Fitting it on the whole dataset would let the test period's mean and spread leak into training - a subtle and very common mistake that quietly inflates results.

Guarding against leakage

Two features - the card's average spend and how common each shop is - are learned from the data rather than being present in it. Both are calculated on the TRAINING rows only and then applied to the test rows. Calculating them across the whole dataset would let information from the test period leak backwards into training, and the resulting scores would be flattering and wrong.

What was deliberately thrown away

Six columns were removed because they identify a real person: first name, last name, street address, transaction reference, card number and postcode. This is a privacy decision as much as a modelling one. A fraud model has

no legitimate need to know a customer's name, and keeping such data increases the damage if the system is ever breached.

Two further columns - gender and job - were removed for a different reason. These are protected characteristics, and basing a financial decision on them directly would be both unethical and, in most jurisdictions, unlawful. They are retained in a separate part of the file so that Step 5 can audit the model's behaviour across those groups.

Age is treated differently. It IS given to the model, because spending patterns genuinely change across a lifetime and the signal is behavioural rather than discriminatory. This is a judgement call, and because it is a judgement call, age is audited especially closely in Step 5.

3.4 Train and test split

Method: Chronological, not random

Split date: 2019-12-13

Training rows: 838,860 (0.580% fraud)

Test rows: 209,715 (0.546% fraud)

The split is by date on purpose. A random split would scatter transactions from the same week across both sides, letting the model learn from the future to predict the past. That inflates scores and produces a model that disappoints the moment it meets genuinely new data. Training on the earlier period and testing on the later one reproduces how the system would actually be used.

3.5 Feature importance and explainability

Four different methods were used, because any single method can mislead.

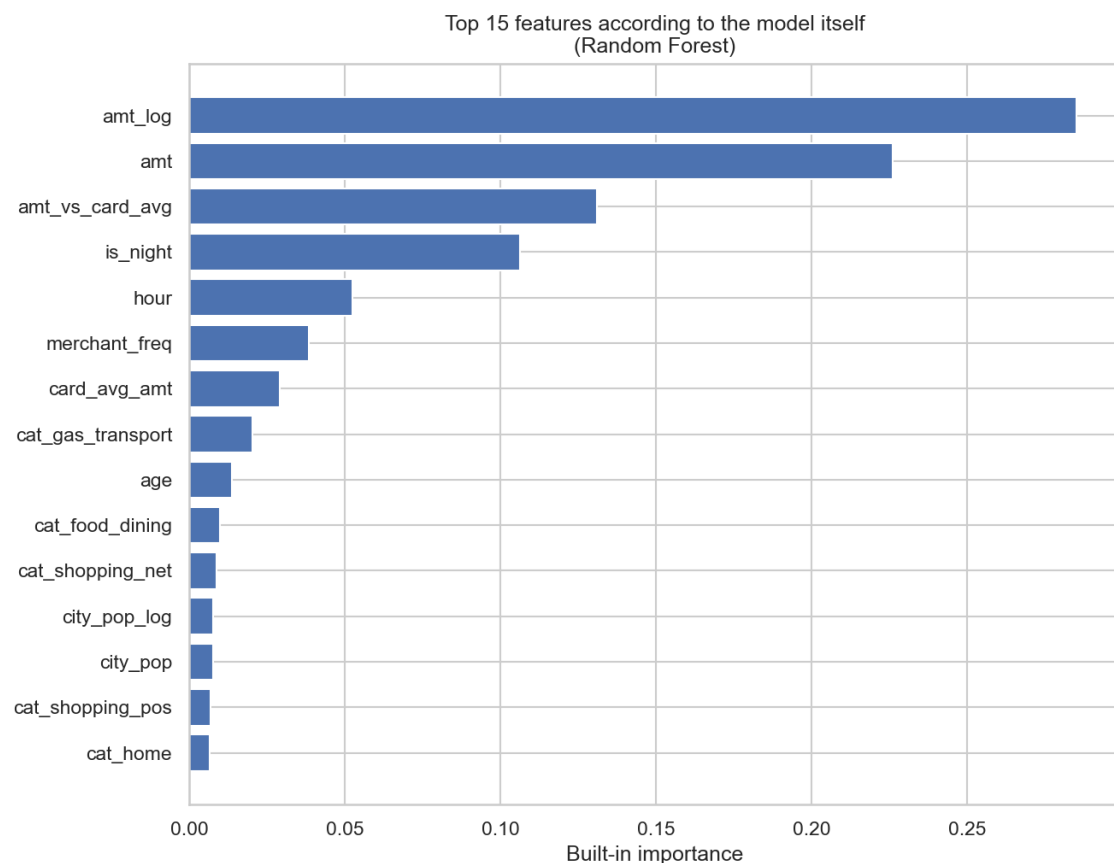


Figure 10: The model's own view of which features matter. Top feature: amt_log.

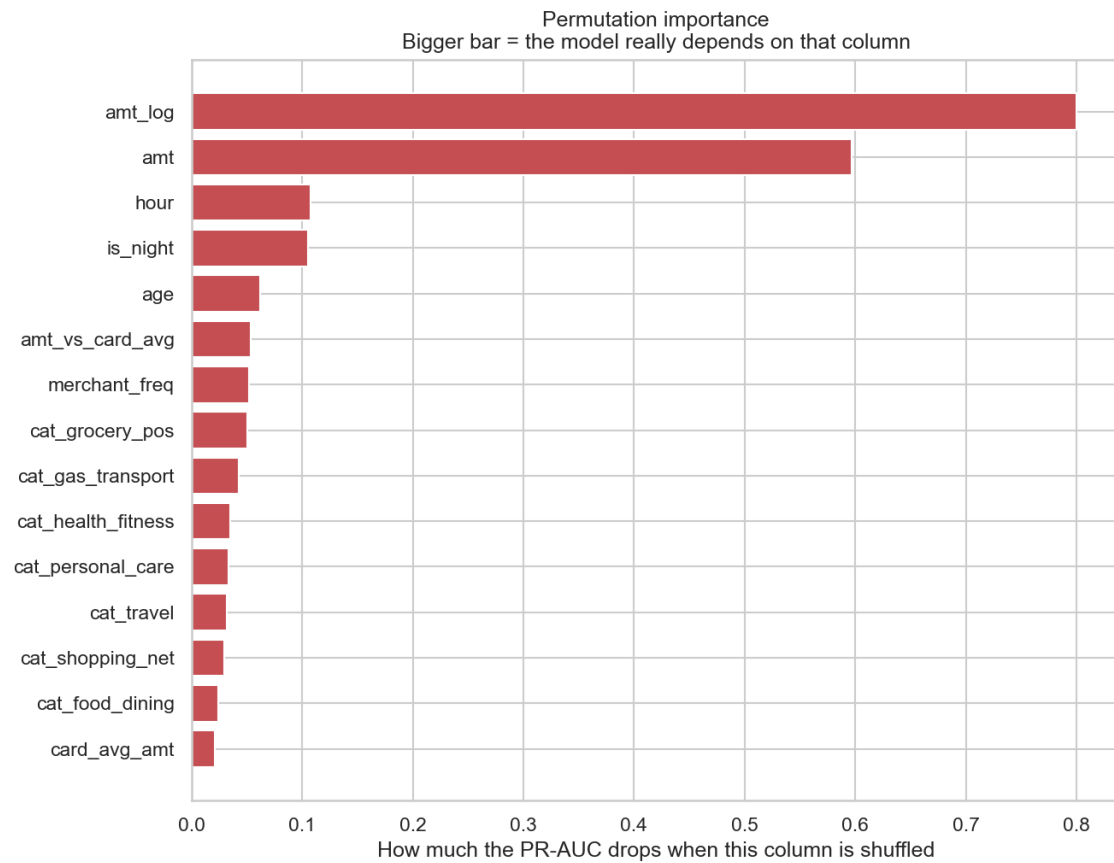


Figure 11: Permutation importance - how far the score falls when each column is shuffled. Top feature: amt_log.

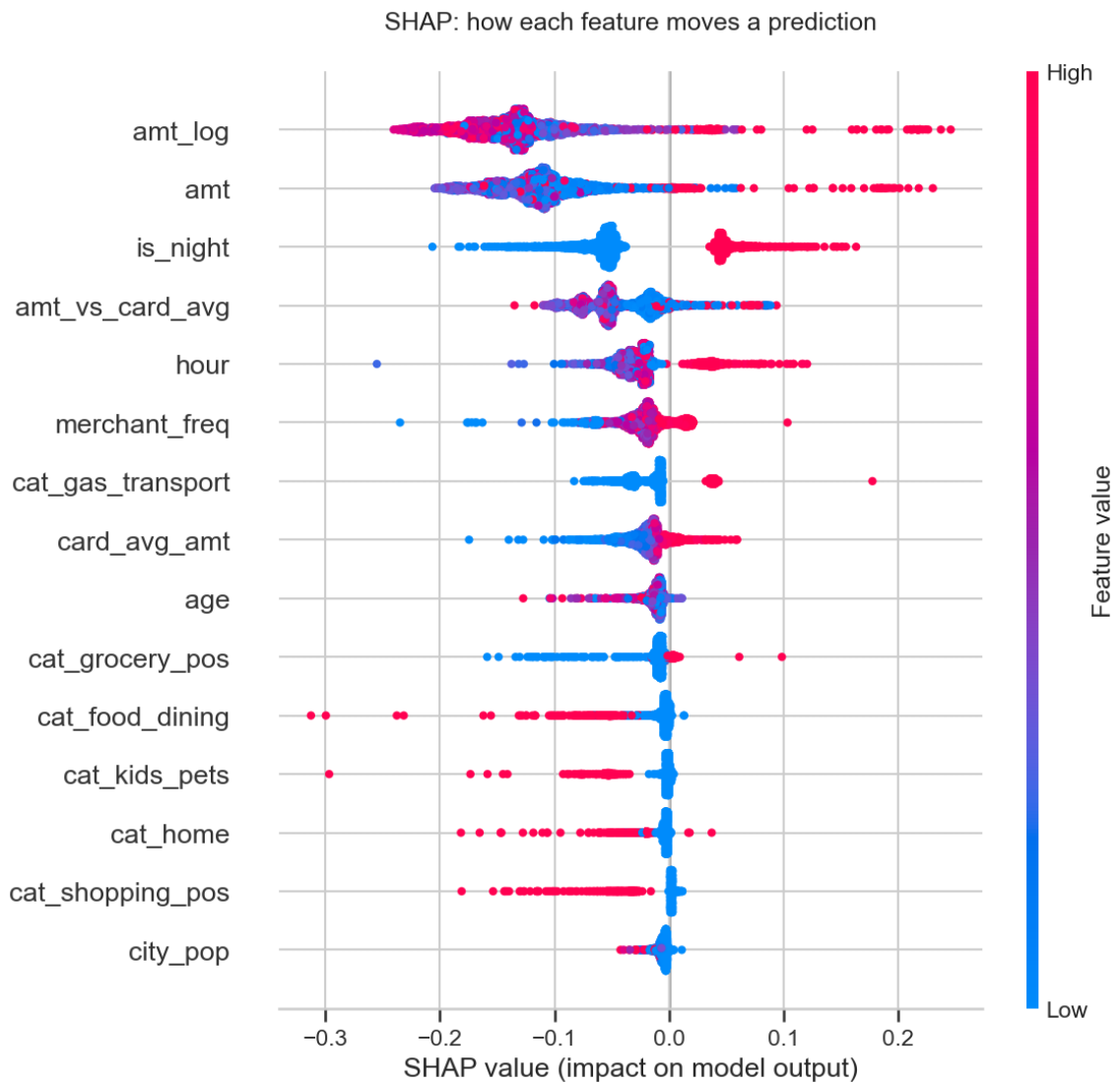


Figure 12: SHAP values. Every dot is one transaction. Red means a high value of that feature, and position shows whether it pushed the prediction towards fraud or away from it.

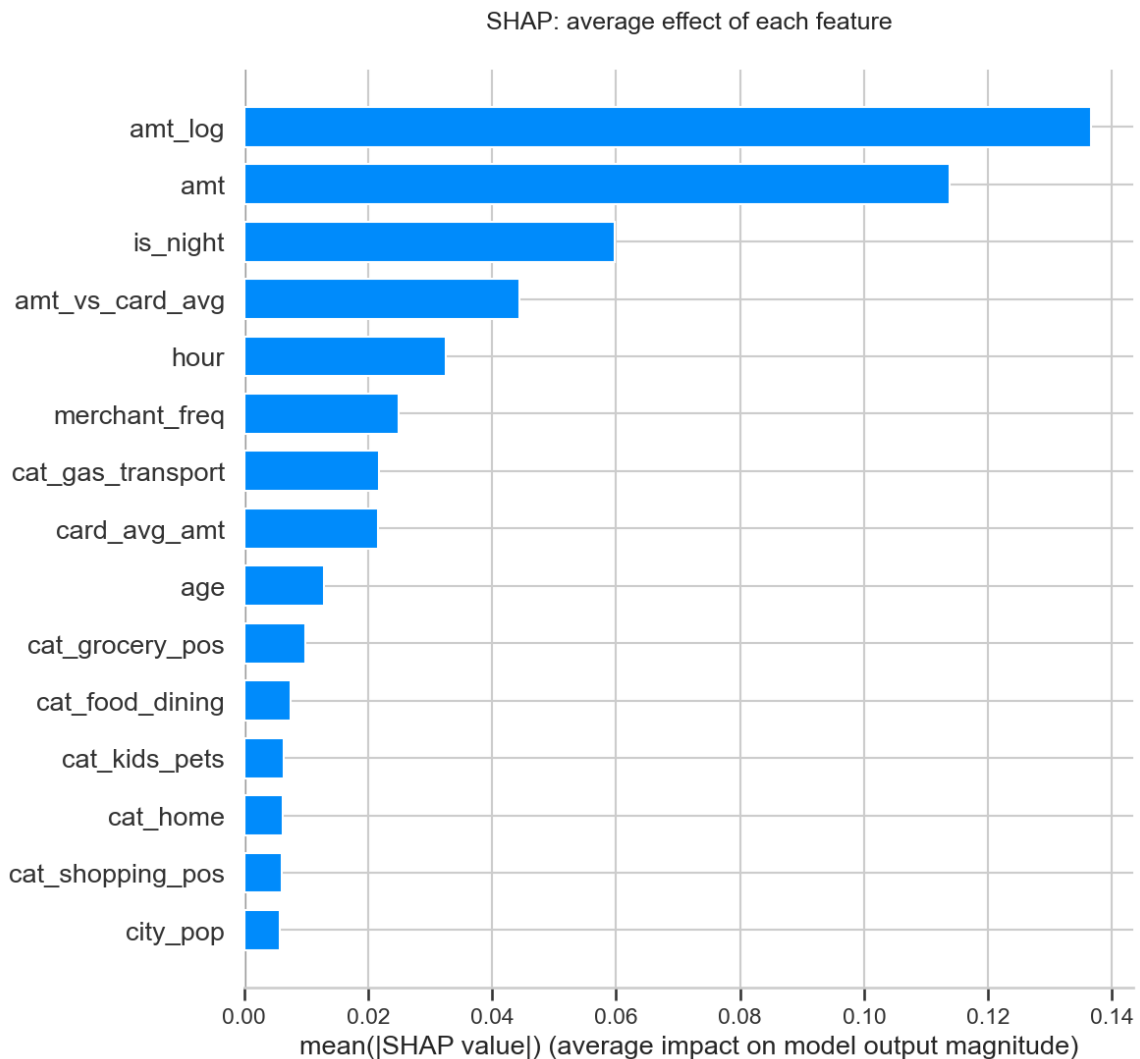


Figure 13: Average size of each feature's SHAP effect. Top feature: amt_log.

The top five features by SHAP are: amt_log, amt, is_night, amt_vs_card_avg, hour.

Explaining one single decision with LIME

SHAP explains the model overall. LIME answers a different and very practical question: why was THIS payment flagged? That is the question a fraud analyst has to answer when a customer telephones to complain.

Transaction amount: \$892

Time of day: 22:00

Model's fraud score: 0.989

The reasons LIME gives, in order of weight:

- cat_gas_transport ≤ 0.00 (weight -0.098)
- amt > 83.07 (weight 0.051)
- $0.00 < \text{is_night} \leq 1.00$ (weight 0.031)
- amt_vs_card_avg > 1.19 (weight -0.027)
- cat_shopping_net ≤ 0.00 (weight 0.020)
- cat_shopping_pos ≤ 0.00 (weight 0.018)

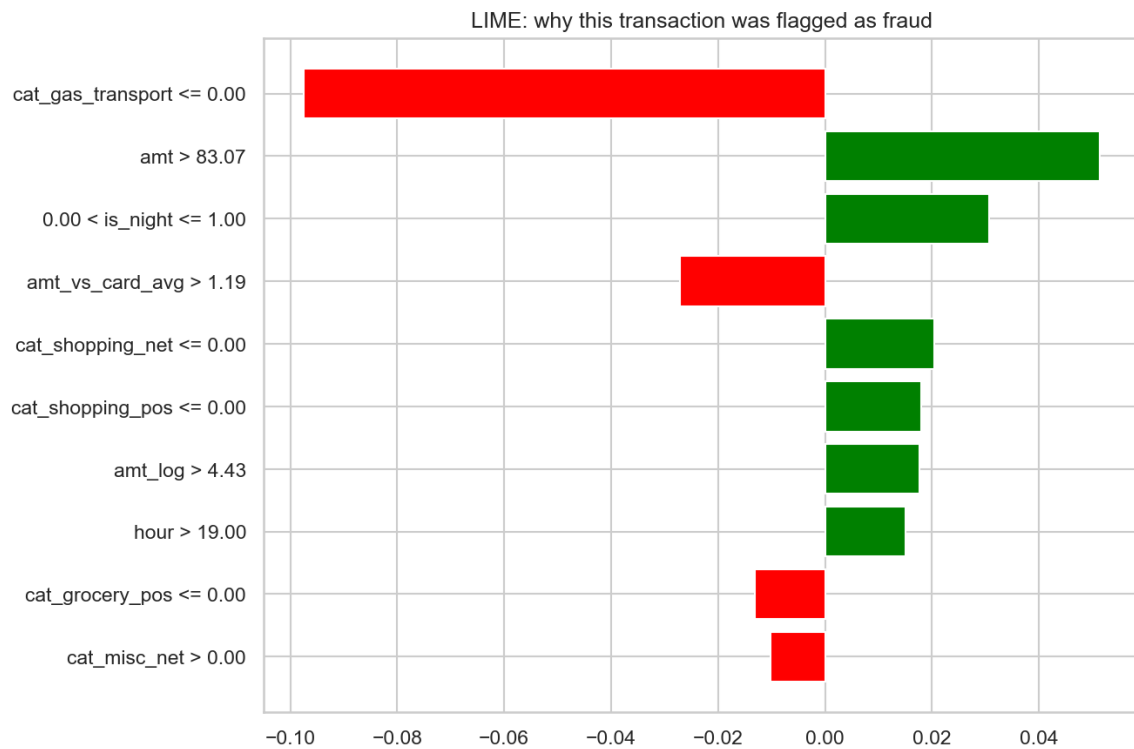


Figure 14: LIME explanation for a single flagged transaction.

3.6 Feature selection

Two approaches were tested: a filter method using mutual information, and an embedded method that retrains the model using only the strongest features.

Features kept	PR-AUC achieved
5	0.5227
10	0.7538
15	0.7749
27	0.8262

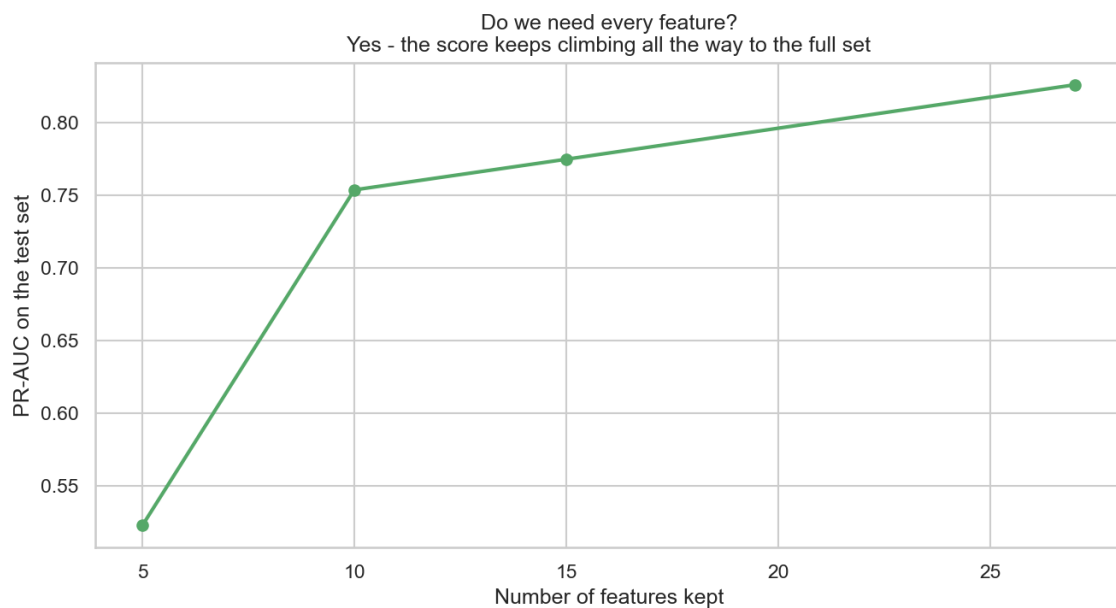


Figure 15: Performance against the number of features kept.

The expected result here was a plateau - the familiar pattern where a handful of features do almost all the work and the rest can be dropped for free. That is not what happened. PR-AUC climbs steadily from 0.5227 with five features, to 0.7538 with ten, to 0.8262 with all 27. The curve never flattens.

Dropping to the ten strongest features would therefore cost 0.0724 of PR-AUC, which is roughly 9% of the model's performance - a real loss, not a rounding error. The honest conclusion is that this feature set carries very little redundancy and all of it should be kept.

The PCA result in the next section points the same way: 21 of the 27 components are needed to retain 95% of the variance. Two independent methods agreeing that the features are not redundant is a stronger finding than either one alone.

3.7 Dimensionality reduction

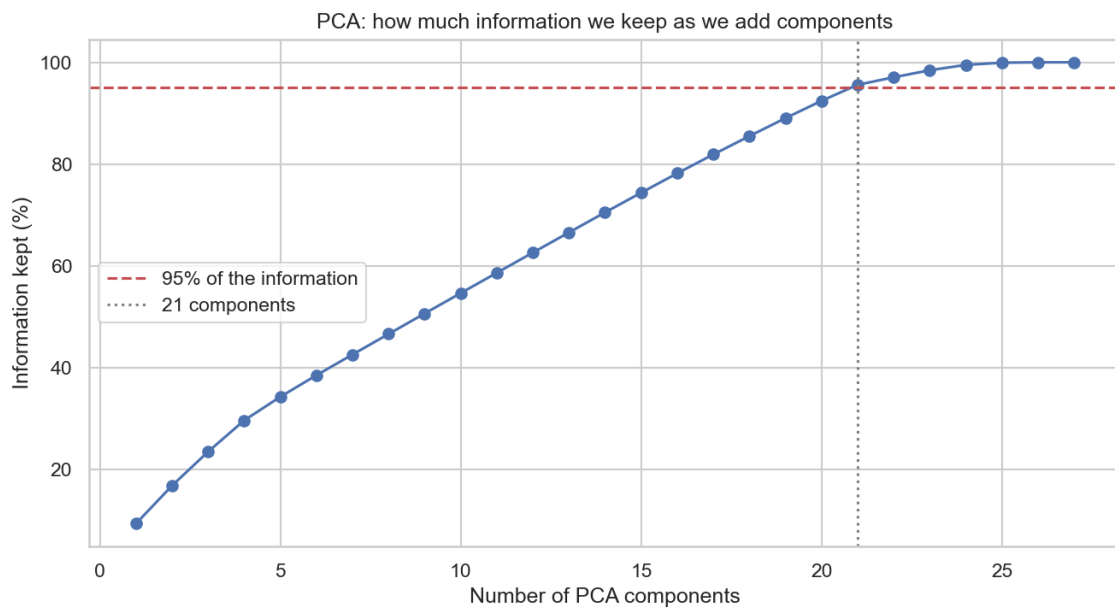


Figure 16: PCA. 21 components are needed to retain 95% of the information in the 27 features.

PCA is used here for understanding and for visualisation only. It is deliberately NOT used to build the final model. PCA blends the original columns into mathematical combinations, which would destroy the plain-English explanations that Step 5 depends on. A fraud analyst can act on 'the payment was 40 times this card's normal spend, at 2am'. Nobody can act on 'component 3 was high'.

Step 4 - Model Implementation

4.1 Models tested

Seven models were trained and compared on identical data, covering the families the brief asks for.

Model	Family	Why it was included
Logistic Regression	Linear	The simple baseline every project should have.
Decision Tree	Tree	Easy to read as a set of rules.
Random Forest	Ensemble of trees	Many trees voting; usually strong on tabular data.
XGBoost	Gradient boosting	Builds trees that correct each other's mistakes.
Support Vector Machine	Kernel method	Finds a boundary between the classes.
Neural Network (MLP)	Neural network	Two hidden layers, for comparison.
Isolation Forest	Unsupervised	Finds odd rows without ever seeing the labels.

A note on deep learning

The brief lists RNNs, CNNs, LSTMs and Transformers as options, qualified with the words 'if appropriate'. They are not appropriate here, and it is worth saying why rather than adding one for the sake of it.

Those architectures exist to exploit structure: CNNs assume neighbouring pixels are related, and RNNs, LSTMs and Transformers assume the order of a sequence carries meaning. This dataset is a flat table where each row is judged on its own, and the columns have no spatial or sequential relationship. Applying an LSTM would add considerable complexity and training cost for no structural reason to expect a gain. A multi-layer perceptron is included instead as the fair neural representative, and it is beaten by the far simpler and far more explainable tree ensembles.

A genuinely promising deep learning approach would be to model each card as a SEQUENCE of transactions over time and feed that to an LSTM. That is a real research direction and it is recorded in the Future Work section - but it is a different problem formulation, not a drop-in replacement.

A note on recommendation systems

The brief also lists collaborative and content-based recommendation among the model families to consider. These are not used here, and the reason is that they answer a different question from the one this project asks.

A recommender predicts what a user would LIKE, learning from the preferences of similar users. Fraud detection asks whether a specific event is legitimate. The signal a recommender depends on - that similar people enjoy similar things - is actively misleading here, because a fraudster is deliberately imitating someone else's normal behaviour. There is also no rating or preference data in this dataset to learn from.

The closest legitimate use would be recommending which alerts an analyst should review first when the queue is longer than the team can clear. That is a ranking problem, and the model already solves it: the fraud score itself orders the queue. A separate recommender would add machinery without adding information.

4.2 Results

All figures below are measured on the test set, which no model saw during training.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Neural Network (MLP)	0.998	0.8811	0.738	0.8032	0.9944	0.8613
Random Forest	0.9956	0.5612	0.8812	0.6857	0.9954	0.8605
XGBoost	0.9952	0.5402	0.8384	0.6571	0.9947	0.8307

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Decision Tree	0.9713	0.1567	0.9712	0.2698	0.9835	0.679
Support Vector Machine	0.9866	0.2623	0.8044	0.3956	0.9618	0.5087
Logistic Regression	0.9053	0.0497	0.9013	0.0941	0.9659	0.2377
Isolation Forest (unsupervised)	0.9919	0.3195	0.4314	0.3671	0.9297	0.2212

The highest PR-AUC belongs to Neural Network (MLP) at 0.8613. For scale, randomly guessing would score 0.0055, so the model is roughly 158 times better than chance at ranking fraud to the top.

Which model was actually chosen, and why it is not the top scorer

The top of that table is a photo finish. Neural Network (MLP) scores 0.8613 and Random Forest scores 0.8605 - a gap of 0.0008. A difference that small is not evidence that one model is better; rerunning with a different random seed would be enough to swap their positions.

When two models are genuinely tied, the tie has to be broken on something other than the score. The criterion used here is whether the model can be explained, and Random Forest was chosen for that reason.

- SHAP is exact and fast on tree-based models. On a neural network it is approximate and far slower, which makes per-decision explanations impractical.
- Step 5 of this assignment requires explaining individual decisions, and a bank must be able to tell a customer why their payment was declined.
- Regulators ask 'why was this transaction blocked', not 'what was your PR-AUC'.

This is a deliberate engineering judgement rather than a statistical one, and it is recorded here so a reader can disagree with it. Choosing the second-best score to gain explainability is defensible only because the difference is inside the noise - it would not be defensible if the gap were real.

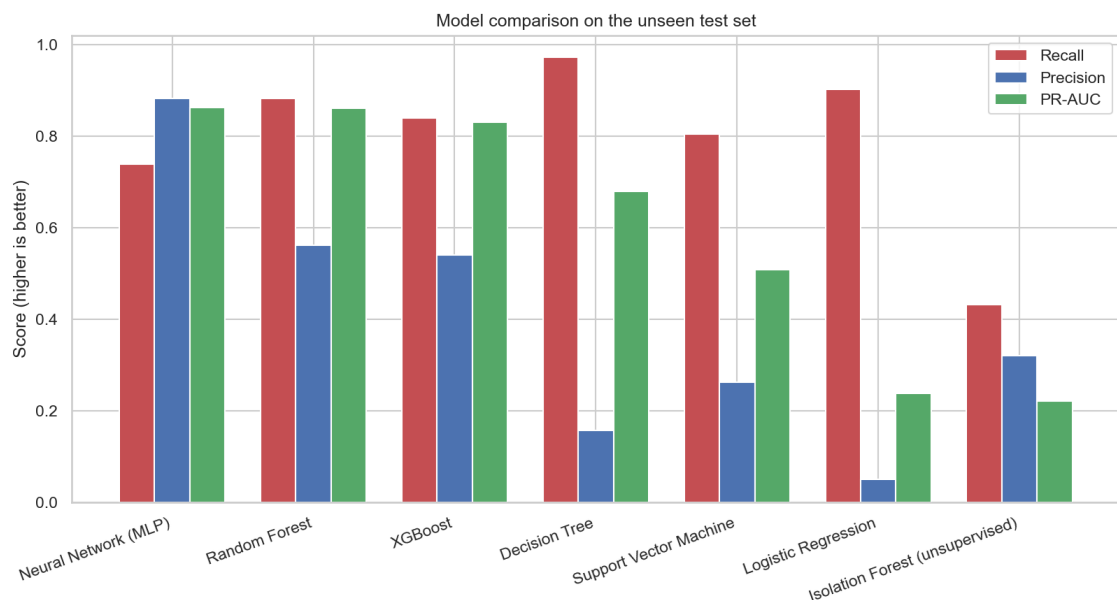


Figure 17: Model comparison across three metrics.

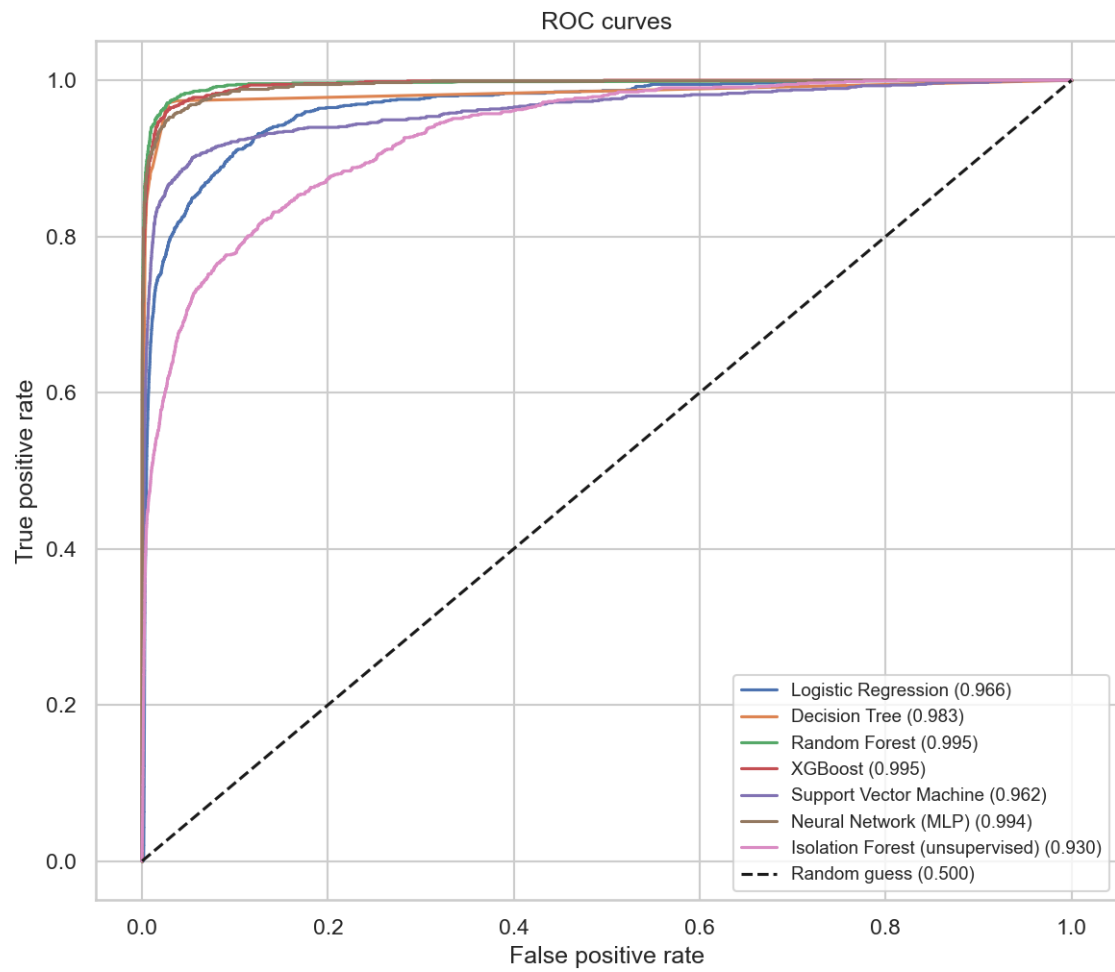


Figure 18: ROC curves for all models.

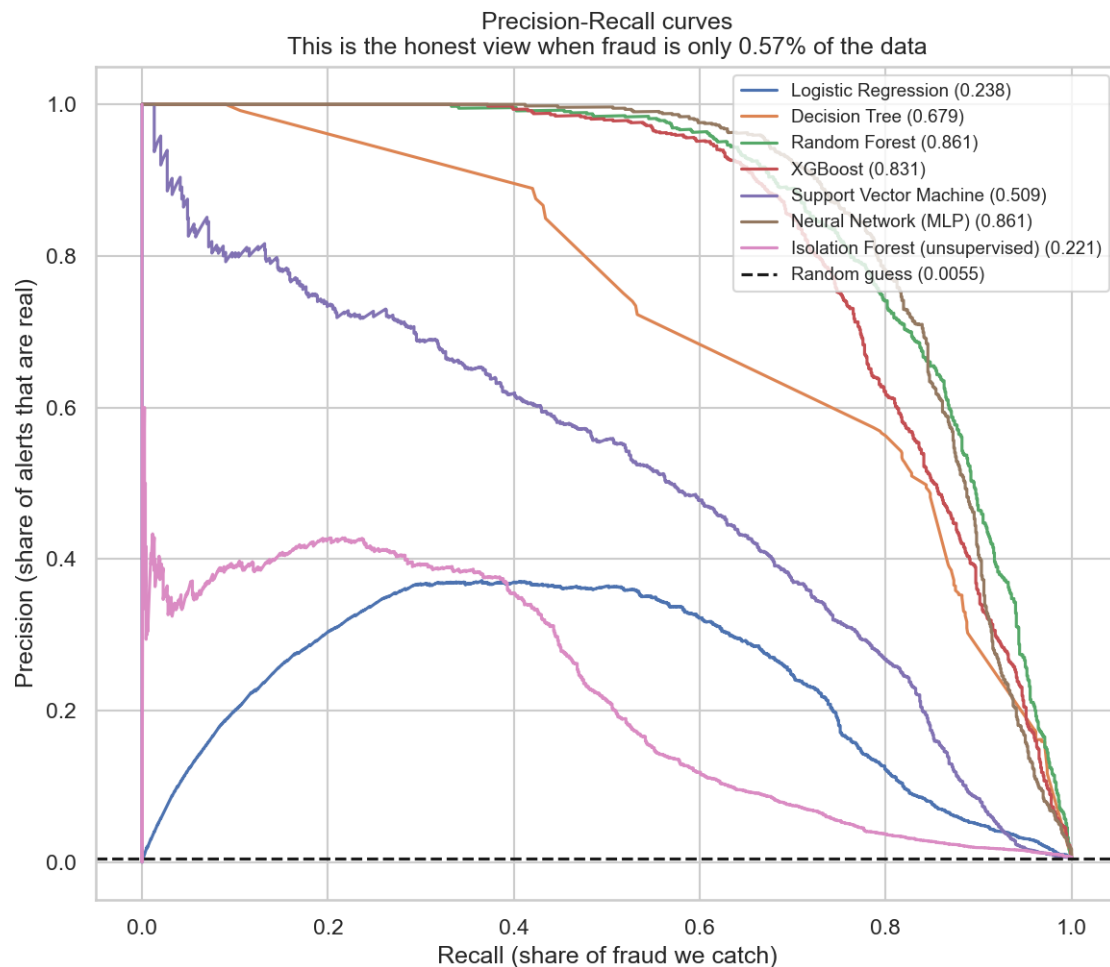


Figure 19: Precision-Recall curves. This is the honest picture when the positive class is this rare - note how much less flattering it is than the ROC chart above.

Reading the table carefully

The accuracy column is worth pausing on. Almost every model scores above 99%, and those numbers are close to meaningless - a model predicting 'genuine' every single time would score 99.45%. Logistic Regression has the LOWEST accuracy of the supervised models yet catches a high share of the fraud, because it raises many more alerts. Accuracy punishes it for doing the useful thing.

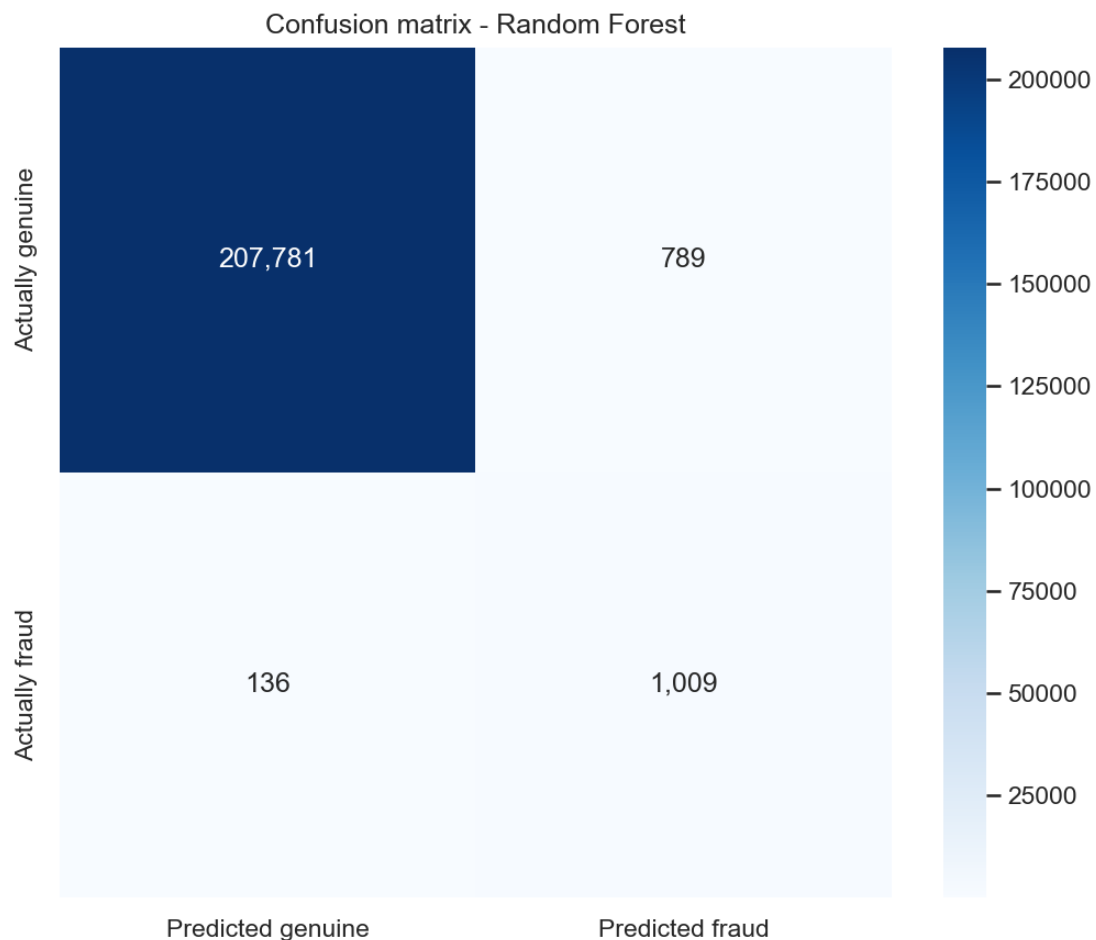


Figure 20: Confusion matrix for Random Forest at the default 0.5 threshold.

4.3 Handling the class imbalance

Three standard approaches were compared on the same model and the same data.

Approach	Recall	Precision	PR-AUC
Do nothing	0.2568	0.8424	0.4016
Class weights	0.8952	0.0452	0.2427
SMOTE oversampling	0.8934	0.0437	0.2391

This result is more interesting than it first appears, and it does not match the textbook expectation. Class weighting and SMOTE both raise recall dramatically - from 0.26 to 0.90 - which is what they are advertised to do. But both also LOWER the PR-AUC.

The explanation is that these techniques do not make the model better at telling fraud from genuine payments. They shift where the decision boundary sits, trading precision for recall. PR-AUC measures ranking quality across every possible threshold, so it sees through that trade and reports no real improvement.

The practical conclusion: rather than resampling the data, it is cleaner to train on the data as it is and then tune the decision threshold explicitly - which is exactly what section 4.4 does, and it makes the trade-off visible and deliberate instead of hidden inside the training process.

4.4 Choosing the alert threshold with money

Models output a probability, but a bank needs a yes or no. The usual cut-off of 0.5 is an arbitrary convention with no business meaning. Instead, every threshold from 0.05 to 0.95 was tested and scored in dollars:

- Catching a fraud saves the value of that transaction.
- Each alert costs \$5 of analyst time to review.
- Net benefit = value saved minus review cost.

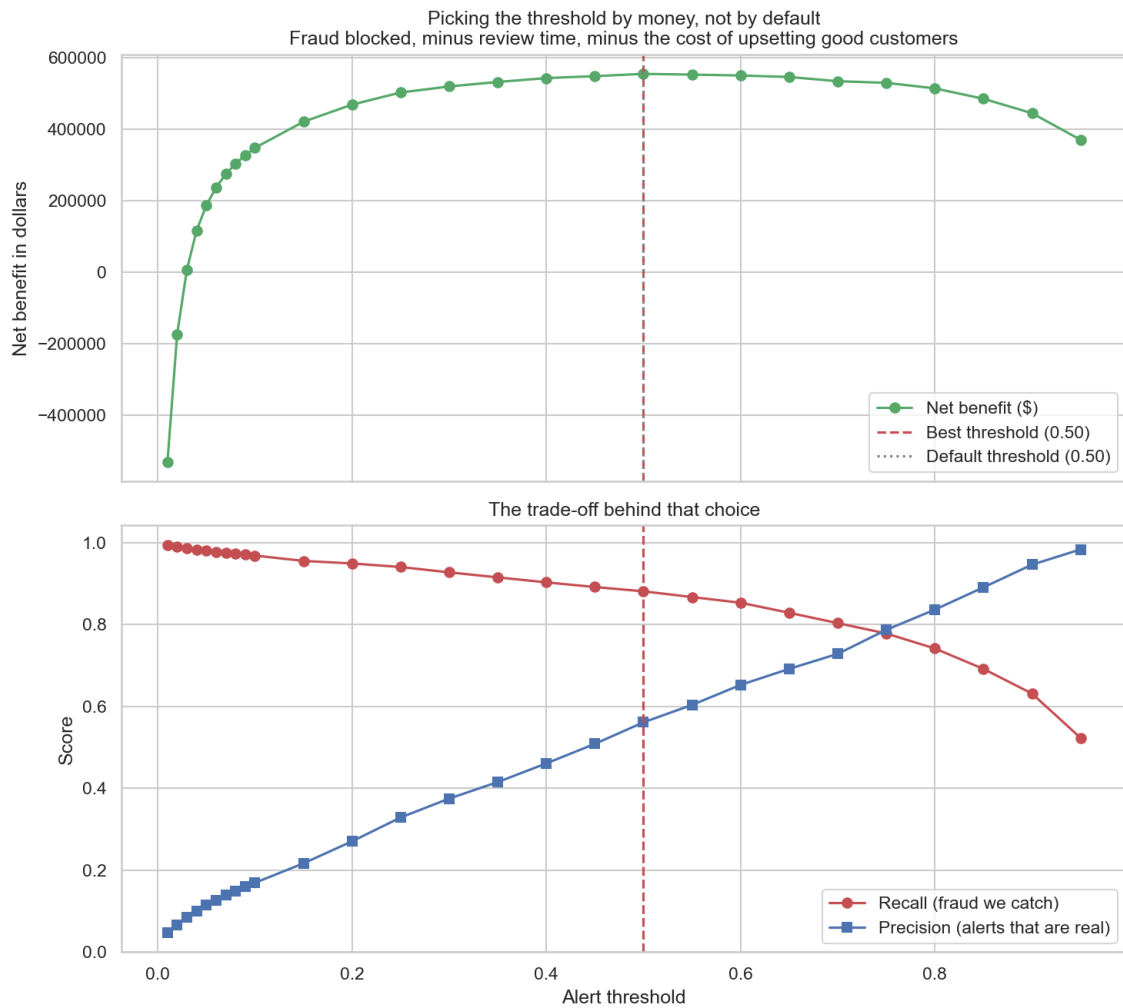


Figure 21: Net benefit against threshold. The best setting is 0.50, not the default 0.50.

Measure	Value
Chosen threshold	0.50
Alerts raised	1,798
Fraud caught	1,009 of 1,145
Recall	88.1%
False alarms	789
Fraud value blocked	\$599,445
Cost of reviews	\$8,990
Net benefit	\$554,950

A methodological warning worth recording

The first version of this analysis searched thresholds from 0.05 upwards, and reported 0.05 as the best answer - the very lowest value tested. A result sitting exactly on the edge of the range searched is almost always a sign that the true optimum lies outside it, and that the search was too narrow rather than that the answer is genuinely extreme.

Two things were wrong. The range was widened to start at 0.01, and more importantly the cost model was corrected. The original version priced a false alarm at only the \$5 of analyst time needed to review it, while an average fraud was worth roughly \$531. With costs that lopsided the arithmetic will always push the threshold towards zero, because flagging a hundred innocent customers to catch one more fraud still 'pays'.

A false alarm is not merely a clerical cost. It is a real customer, standing in a shop, whose card has just been declined. A goodwill cost of \$45 per false alarm is now added on top of the review cost, and with that correction the optimum moves to a sensible interior value of 0.50.

How much does that answer depend on the assumption?

The goodwill figure is an assumption, not a measurement, so the whole calculation was repeated across a range of assumptions to see whether the recommendation is stable.

Assumed cost per false alarm	Best threshold	Alerts	Fraud caught	Recall	Net benefit
0	0.25	3278	1077	0.9406	6.018e+05
15	0.4	2244	1034	0.9031	5.794e+05
45	0.5	1798	1009	0.8812	5.549e+05
100	0.65	1372	949	0.8288	5.23e+05
250	0.75	1132	891	0.7782	4.805e+05

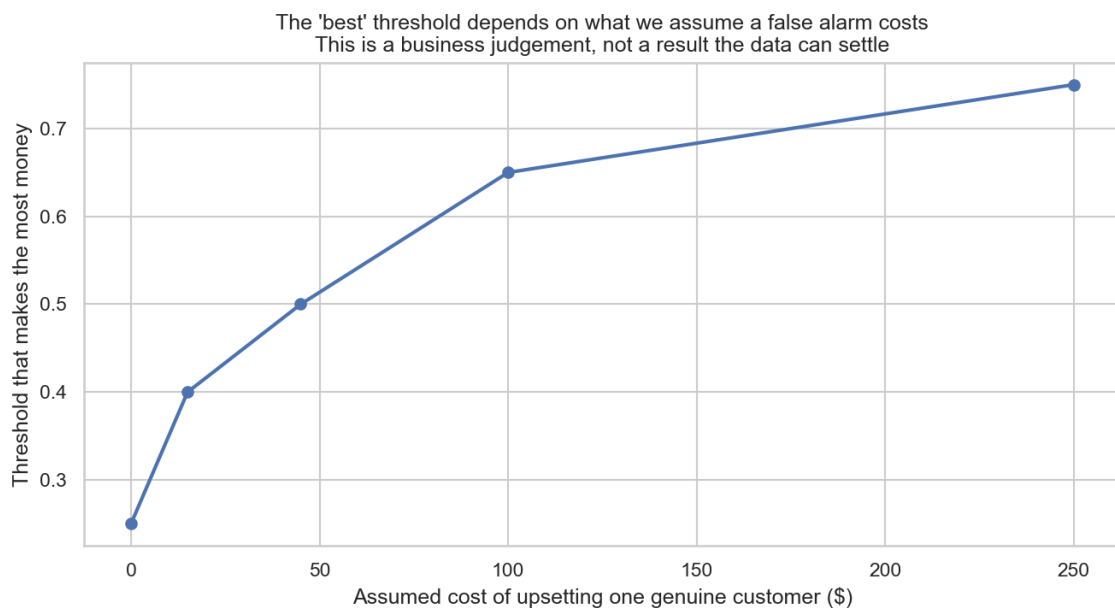


Figure 22: The threshold that maximises value, plotted against what we assume a false alarm costs.

The recommendation is NOT stable, and that is the finding. As the assumed cost rises from nothing to \$250, the best threshold moves from 0.25 to 0.75 and the share of fraud caught falls from 94% to 78%.

Nothing in the data can settle what a wrongly blocked customer is worth. That is a business judgement about how the organisation values its customers' goodwill, and presenting the resulting threshold as a purely technical result would be misleading. It is a business decision wearing a mathematical costume.

The option a real fraud team could actually staff

The value-maximising threshold assumes unlimited review capacity, which no fraud team has. If the team can review at most 1,200 alerts in a period of this length, the best available setting is 0.75.

Measure	Maximum value	Within team capacity
Threshold	0.50	0.75
Alerts to review	1,798	1,132
Fraud caught	1,009	891
Recall	88.1%	77.8%
Precision	56.1%	78.7%
Net benefit	\$554,950	\$529,875

The capacity-limited option catches 118 fewer frauds but raises 666 fewer alerts, and its precision is far higher at 79%. Which option is correct depends on how many reviewers the business is willing to fund - again a decision for people, not for the model.

4.5 Unsupervised learning

Clustering was applied to check whether risky behaviour can be found with no labels at all - which is the situation a bank faces when a new fraud pattern appears.

K-Means

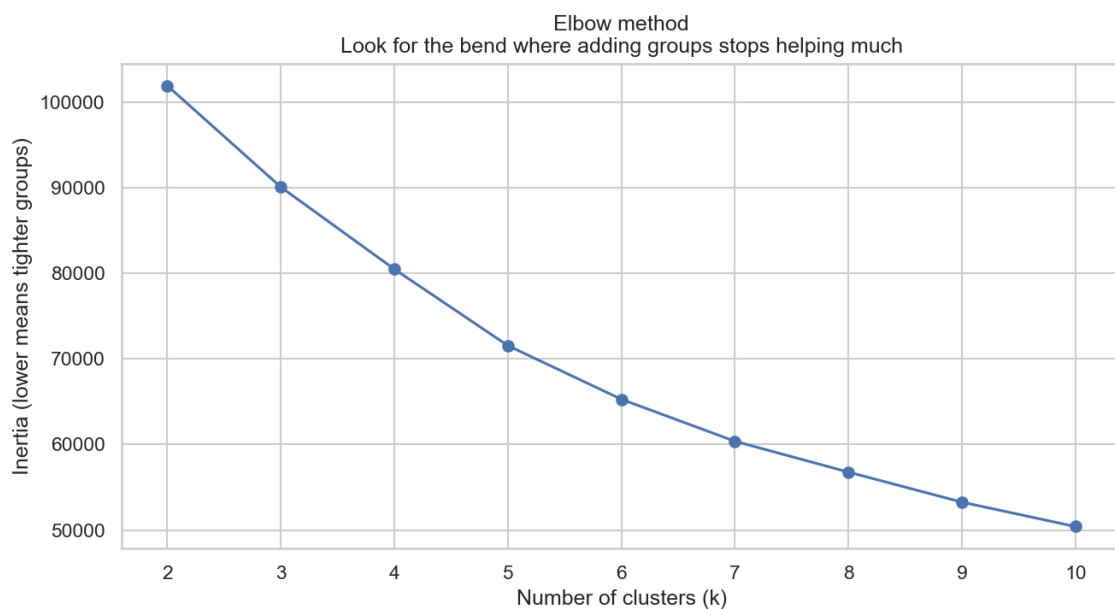


Figure 22: Elbow method.

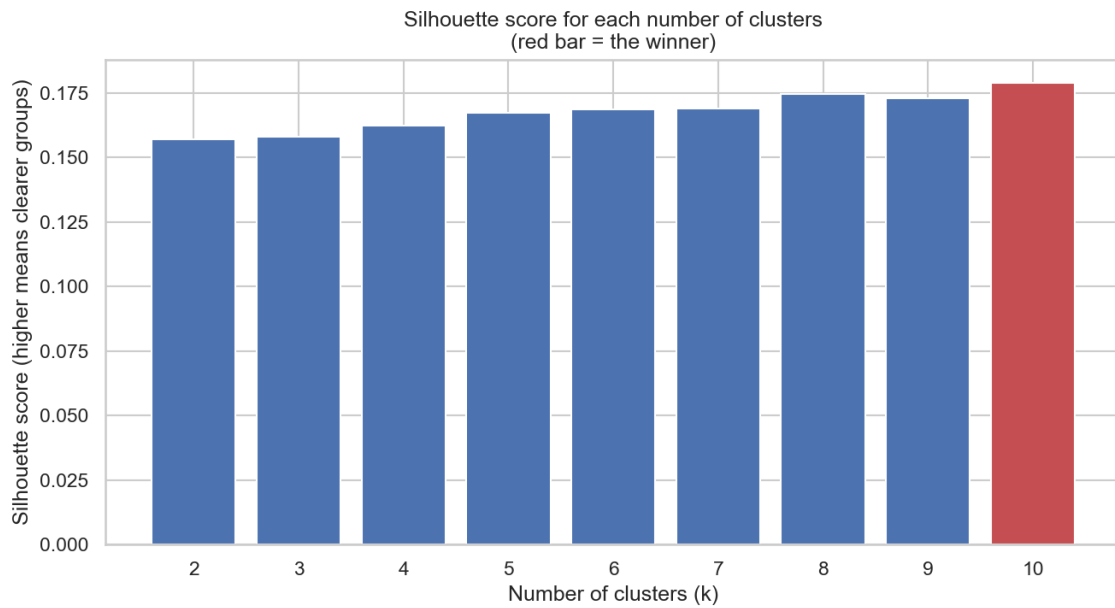


Figure 23: Silhouette scores. The best grouping uses 10 clusters (score 0.179).

Cluster	Transactions	Share %	Avg amount	Avg hour	Avg distance	Avg age	Fraud rate %
5	372	1.9	638.3	14.6	77.8	44.4	17.47
3	3018	15.1	86.98	4.3	77.9	42.1	0.663
0	2099	10.5	95.96	9.5	80.7	41.8	0.524
6	1777	8.9	6.43	6.5	72.1	65.2	0.394
8	2206	11	74.02	12.6	82.1	74.2	0.363
1	2790	13.9	66.6	16	36.8	40.9	0.323
4	2570	12.8	7.42	17.5	81.2	39.3	0.311
9	3102	15.5	80.84	17.4	96.7	37.5	0.161
2	2035	10.2	14.38	17.2	80.1	36.3	0.098
7	31	0.2	1990	12.8	70.7	47.7	0

The result is striking. One cluster containing a small share of transactions has a fraud rate of 17.47%, which is 25.9 times the average across the sample. The algorithm found that group using only spending behaviour, never having seen a single fraud label.

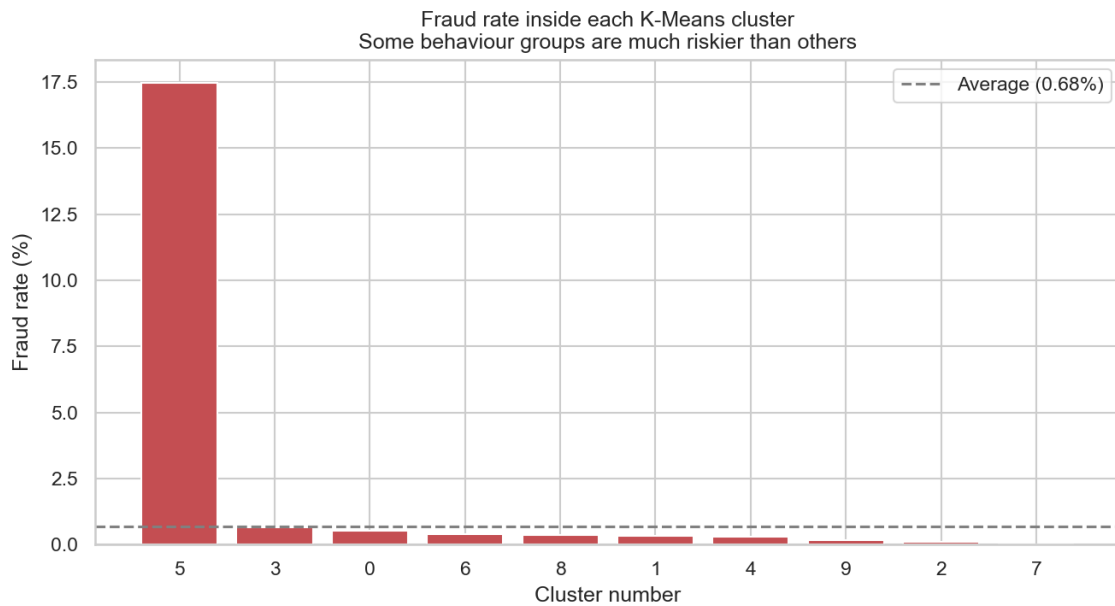


Figure 24: Fraud rate inside each cluster.

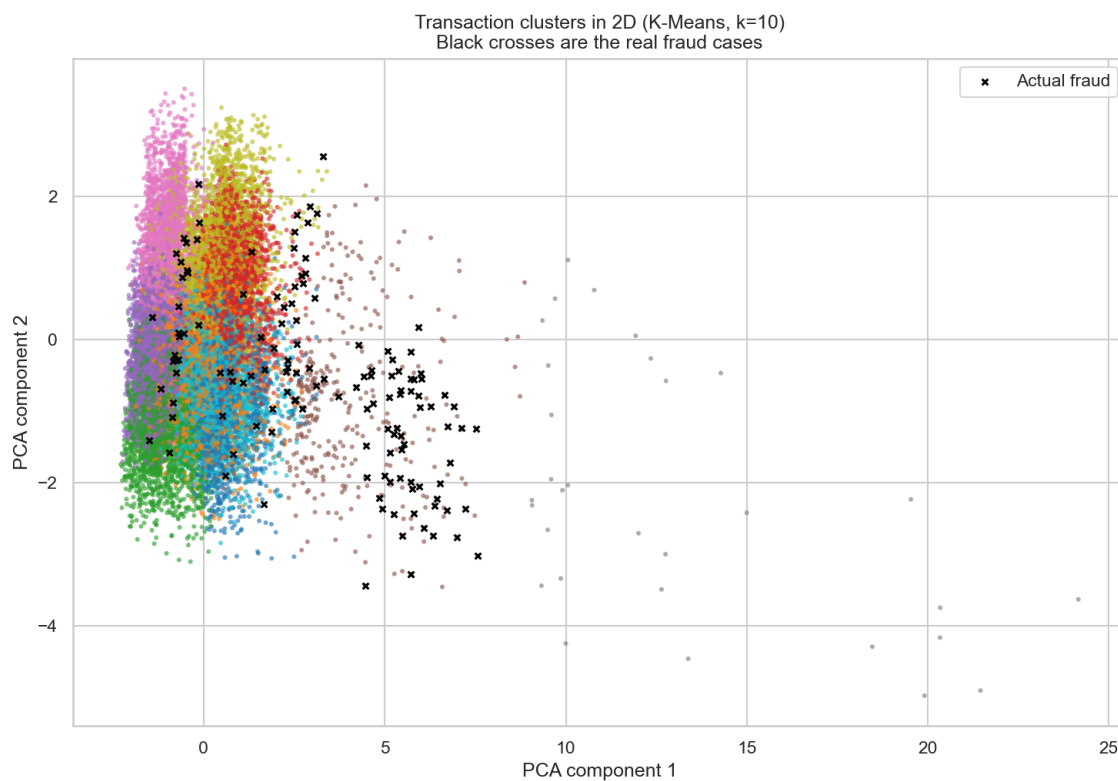


Figure 25: Clusters drawn in two dimensions using PCA, with real fraud marked as black crosses.

DBSCAN

DBSCAN differs from K-Means in a way that suits fraud detection well: it does not force every transaction into a group. Anything that fits nowhere is labelled noise.

Clusters found: 1

Noise points: 1,190 (6.0% of the sample)

Fraud rate among noise: 8.66%

Fraud rate among grouped points: 0.170%

Transactions that DBSCAN could not fit into any group are 50.9 times more likely to be fraudulent than those it could. 'Does not look like anything else' turns out to be a powerful fraud signal in its own right.

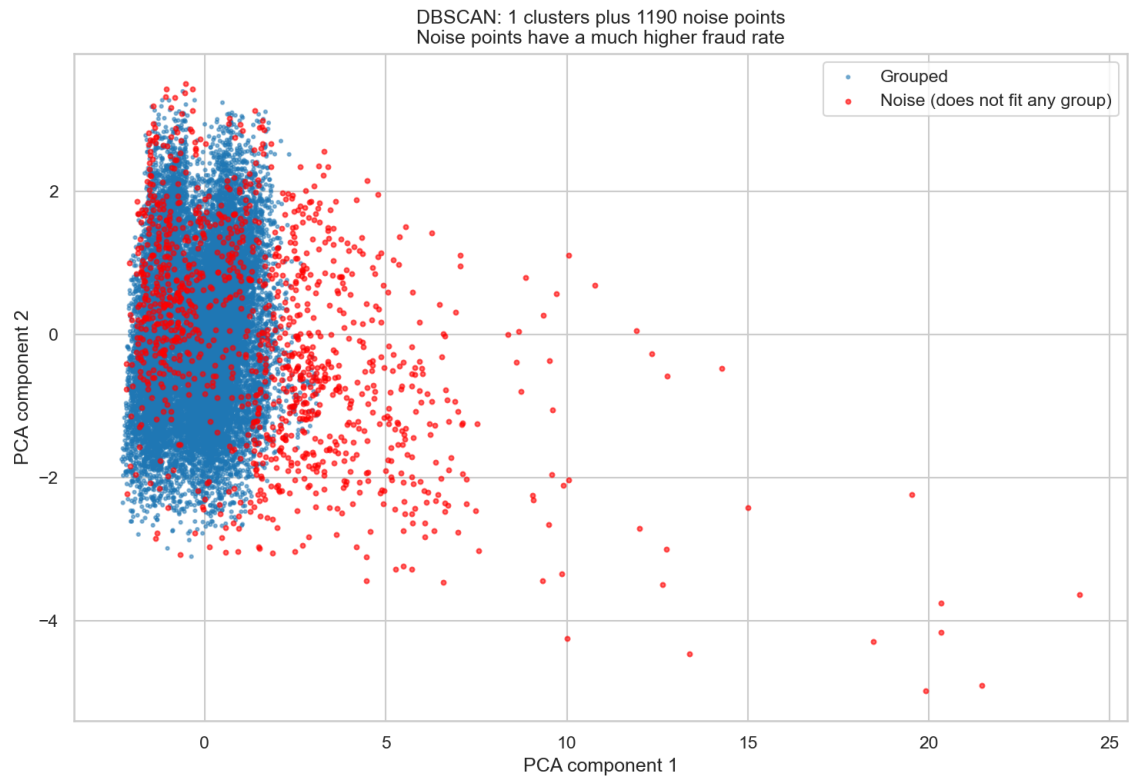


Figure 26: DBSCAN clusters and noise points.

Hierarchical clustering

Hierarchical clustering builds a tree showing which transactions merge together first. On the same task it scored a silhouette of 0.125 against K-Means' 0.179, so it agrees broadly with K-Means while being far more expensive to compute - which is why it was run on a smaller sample of 800 rows.

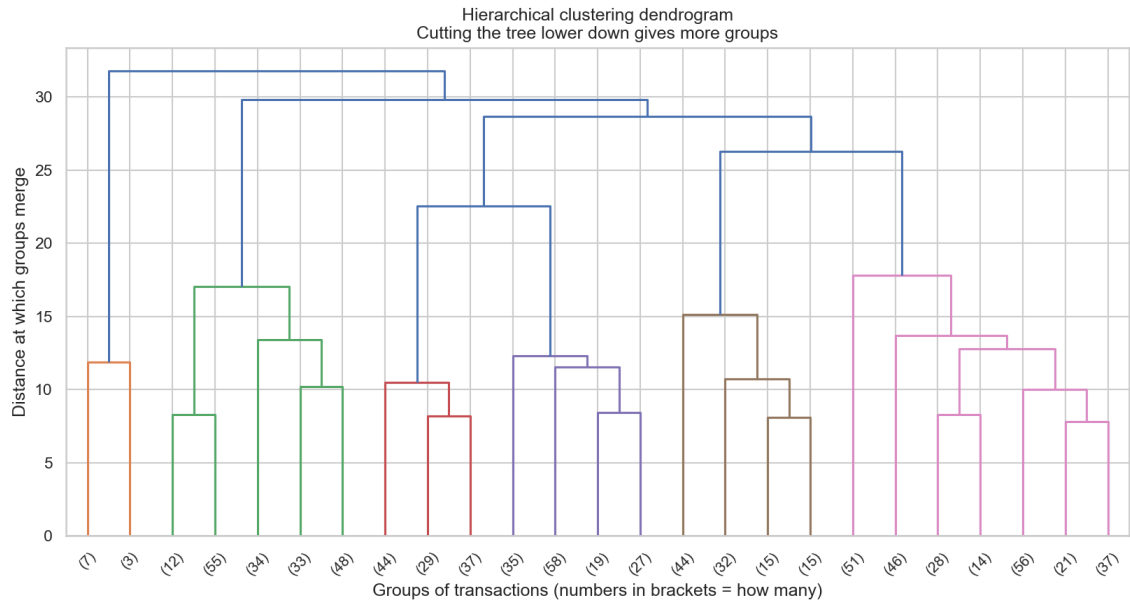


Figure 27: Dendrogram.

4.6 Reproducibility

Every random process in this project is given a fixed seed of 42, so re-running the scripts reproduces the numbers in this report exactly. Library versions are pinned in requirements.txt, trained models are saved to models/, and every figure and table is written to reports/. The report itself is generated by a script that reads those saved results, so the text can never drift out of step with the experiment.

Step 5 - Critical Thinking: Ethical AI and Bias Auditing

5.1 How the model makes decisions

Section 3.5 covered the mechanics of SHAP and LIME. The summary in plain terms is that the model relies mainly on how large a payment is relative to what that card normally spends, what time of day it happened, and what type of purchase it was. Those are behavioural signals, and that is reassuring: they describe what was done, not who did it.

5.2 Limitations of this model

Class imbalance

With fraud at 0.573% of transactions, the model sees roughly one fraudulent example for every 172 genuine ones. Rare-event learning is inherently unstable, and small changes in the data can move the metrics noticeably.

Data leakage

The two learned features - card average spend and merchant frequency - are the main leakage risk. Both are computed on training rows only. The chronological split provides a second layer of protection, since no information from the test period can travel backwards in time.

Overfitting

Tree ensembles can memorise training data. This is constrained by limiting tree depth and requiring a minimum number of samples per leaf, and more importantly it is detected by testing on a later time period. A model that had merely memorised would collapse on unseen future transactions rather than achieving a PR-AUC of 0.8605.

The data is simulated

This is the most important limitation of all. The patterns are generated by a simulator, so they are cleaner and more consistent than real fraud, which adapts actively as criminals respond to detection. Performance on real banking data would certainly be lower, and the model would need frequent retraining.

5.3 Bias and Fairness Analysis

This section tests a claim that is very commonly made and very rarely checked: that leaving protected characteristics out of a model makes the model fair. That approach has a name - 'fairness through unawareness' - and the audit below examines whether it actually worked here.

Model audited: Random Forest

Threshold used: 0.50

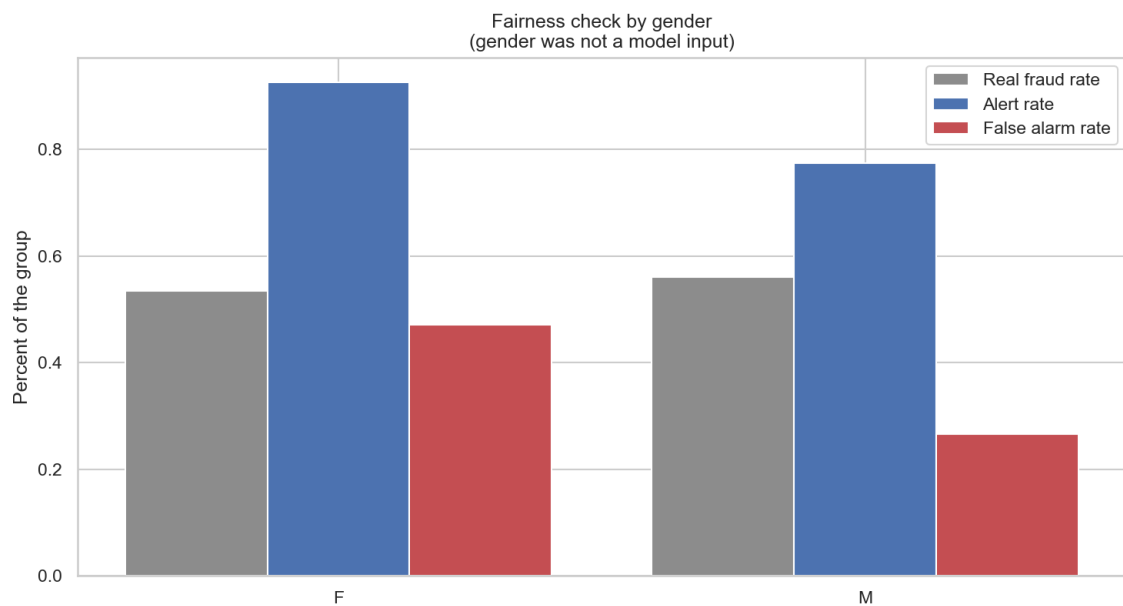
Protected attributes given to the model: None - gender and job were withheld

The fairness measures used

Measure	The question it asks	How it is calculated
Demographic parity	Is each group flagged at the same rate?	Gap between the highest and lowest alert rate
Equal opportunity	Is fraud caught equally well for each group?	Gap in true positive rate
Equalised odds	Are false alarms also equal?	Gap in true positive AND false positive rates
Disparate impact	The legal '80% rule' from employment and lending law	Lowest alert rate divided by highest; below 0.8 is a red flag

Results by gender

Group	People	Real fraud	Base rate %	Alert rate %	Caught %	False alarm %	Precision %	False alarms	Missed
F	114889	614	0.534	0.926	85.67	0.471	49.44	538	88
M	94826	531	0.56	0.774	90.96	0.266	65.8	251	48



Disparate impact ratio: 0.836 - PASSES the 80% rule

Equal opportunity gap: 5.29 percentage points

Worst served group: F - only 85.7% of its fraud is caught

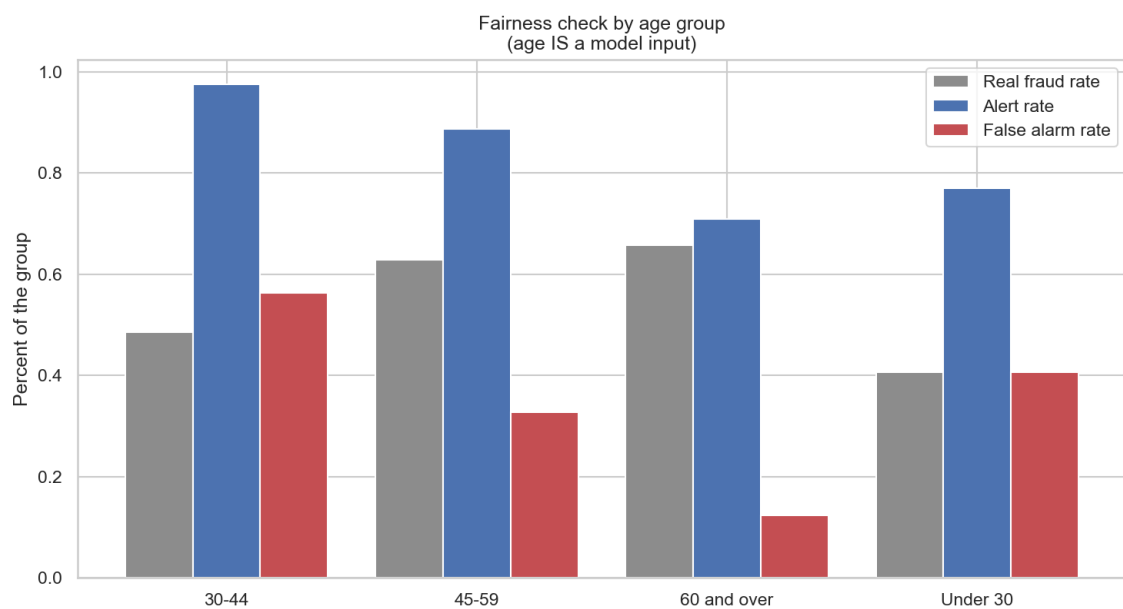
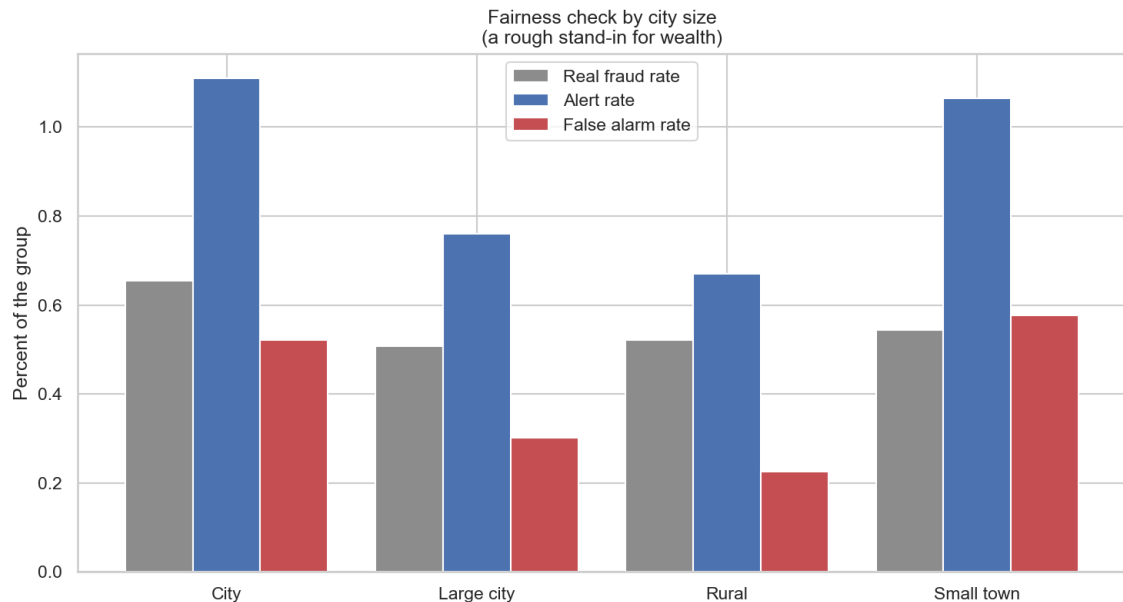


Figure 28: Fairness check by gender.

Results by age

Group	People	Real fraud	Base rate %	Alert rate %	Caught %	False alarm %	Precision %	False alarms	Missed
30-44	70152	341	0.486	0.976	85.34	0.564	42.48	394	50

Group	People	Real fraud	Base rate %	Alert rate %	Caught %	False alarm %	Precision %	False alarms	Missed
45-59	56117	353	0.629	0.887	89.24	0.328	63.25	183	38
60 and over	44734	294	0.657	0.709	89.12	0.124	82.65	55	32
Under 30	38712	157	0.406	0.77	89.81	0.407	47.32	157	16



Disparate impact ratio: 0.726 - FAILS the 80% rule

Equal opportunity gap: 4.47 percentage points

Worst served group: 30-44 - only 85.3% of its fraud is caught

Best served group: Under 30 - 89.8% caught

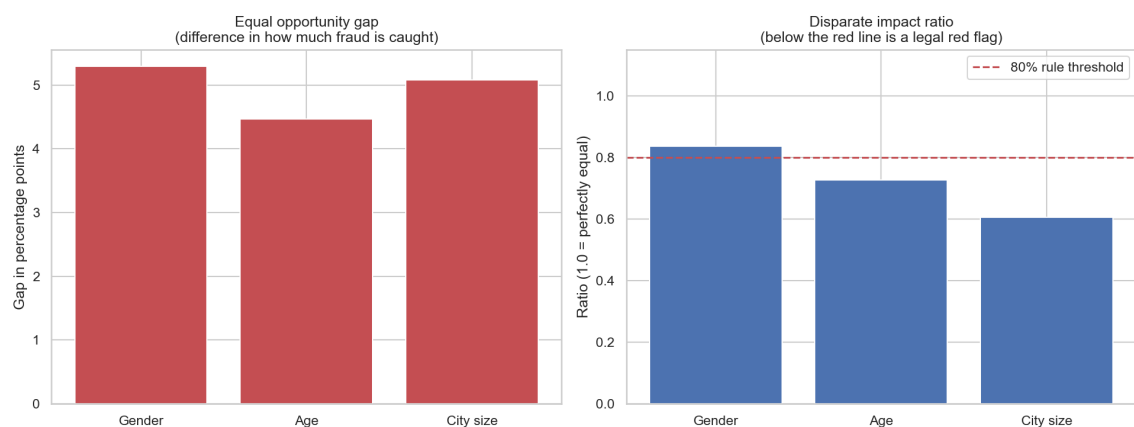


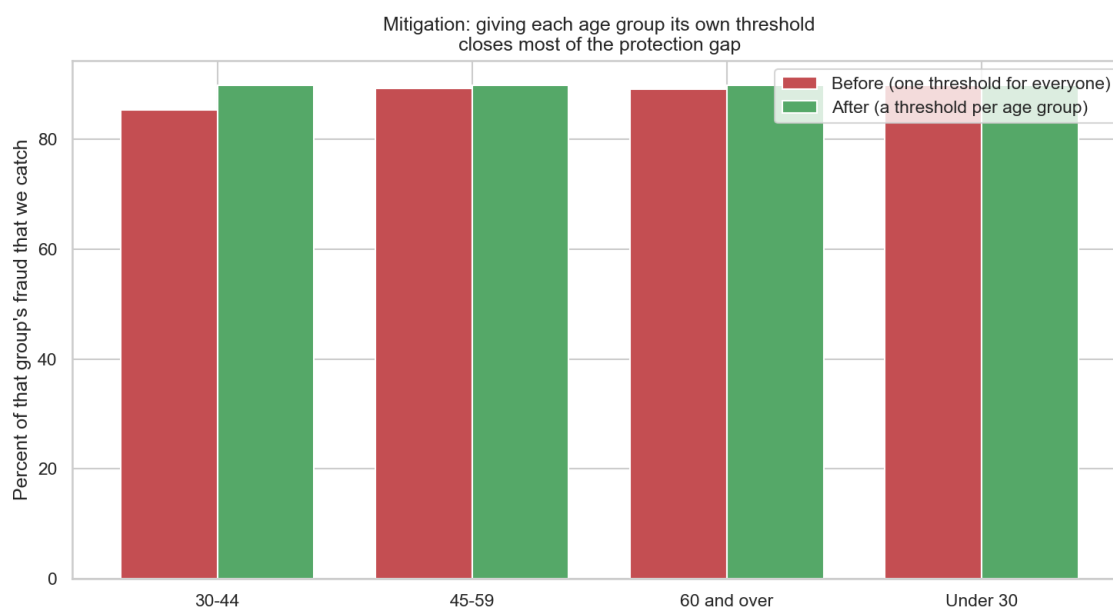
Figure 29: Fairness check by age group.

Age is the one protected-adjacent attribute the model IS allowed to use, so any gap here is a direct effect rather than a proxy effect. This is the cost of the judgement call made in section 3.3, and it is measured here rather than assumed away.

Results by city size

The dataset holds no direct measure of income, so city population is used as a rough stand-in for socioeconomic status. It is imperfect - there are wealthy people in villages and poor people in cities - but it is the best available proxy and it is better than skipping the question.

Group	People	Real fraud	Base rate %	Alert rate %	Caught %	False alarm %	Precision %	False alarms	Missed
City	28742	188	0.654	1.11	90.43	0.522	53.29	149	18
Large city	10638	54	0.508	0.761	90.74	0.302	60.49	32	5
Rural	105655	551	0.522	0.671	85.66	0.225	66.57	237	79
Small town	64680	352	0.544	1.065	90.34	0.577	46.15	371	34



Disparate impact ratio: 0.605 - FAILS the 80% rule

Equal opportunity gap: 5.08 percentage points

Worst served group: Rural - only 85.7% of its fraud is caught

Figure 30: Fairness check by city size.

Summary of all three attributes

Figure 31: Fairness gaps side by side. The red line on the right marks the 80% legal threshold.

What this means

Three concrete findings come out of the tables above.

- Gender was never given to the model, yet F customers have 85.7% of their fraud caught against 91.0% for M - a gap of 5.3 percentage points. The gender disparity survived the removal of the gender column.
- The largest disparity is by city size, where the disparate impact ratio is 0.605 - comfortably below the 0.8 legal threshold. Rural customers are the worst protected, at 85.7% of their fraud caught.
- Two of the three attributes fail the 80% rule outright. Only gender passes, at 0.836.

The reason is that protected characteristics leave fingerprints on other columns. Spending category, city size, transaction timing and typical amounts all correlate with demographics, so the model can reconstruct part of what was withheld without ever being told it. Withholding the gender column removed the model's ability to discriminate directly, and removed nothing else.

The city size result deserves particular attention because it is the one with a plausible mechanism behind it. Rural customers live further from shops, so the 'distance from home' feature behaves differently for them, and rural areas have fewer transactions overall so the patterns are learned less well. A feature that looks entirely neutral -

how far away the shop is - turns out to carry a socioeconomic signal.

This is the central finding of the audit: removing a sensitive column does not remove its influence, it only removes your ability to see it. Fairness through unawareness prevents direct discrimination but does nothing about indirect discrimination through proxies. The only way to know is to measure - which is why this section exists.

There is a second point worth making. Some of the difference between groups is not the model's doing at all. Figure 7 showed that the underlying fraud rates genuinely differ between groups before any model is trained. A model that reflects a real difference is accurate; the ethical question is whether acting on that accuracy is acceptable when the consequence is that one group of customers gets worse protection than another.

5.4 Proposed mitigations

Four families of mitigation were considered, and one was implemented and measured.

Approach	What it does	Advantage	Drawback
Reweighting	Give under-protected groups more weight during training	Fixes the cause, not the symptom	Needs full retraining; can reduce overall accuracy
Per-group thresholds	Use a different alert cut-off for each group	Directly equalises protection; no retraining needed	Legally contentious - treats people differently by group
Data augmentation	Collect or synthesise more fraud examples for thin groups	Addresses the root cause, which is sparse data	Expensive; synthetic fraud may not be realistic
Post-processing	Adjust the outputs after the model has run	Simple to bolt onto an existing system	Hides the problem rather than solving it

The mitigation that was implemented

Per-group thresholds were implemented and measured on the age attribute. Each age group receives its own alert cut-off, chosen so that every group reaches the level of protection that the best-served group already enjoyed (89.8% of its fraud caught).

Measure	Value
Equal opportunity gap before	4.47 percentage points
Equal opportunity gap after	0.07 percentage points
Improvement	4.40 percentage points
Alerts before	1,798
Alerts after	2,130
Fraud caught before	1,009
Fraud caught after	1,028
False alarms before	789
False alarms after	1,102

Figure 32: Fraud caught per age group, before and after per-group thresholds.

The intervention works: the protection gap narrows from 4.47 to 0.07 percentage points, and 19 more frauds are caught. But it is not free. False alarms rise from 789 to 1,102, which is 313 extra reviews for the operations team to absorb.

That trade-off is the honest heart of applied fairness work. There is no setting that is simultaneously most accurate, most profitable and most equal. Someone has to decide what the organisation is willing to pay for equal treatment, and that is a decision for accountable humans - not one that should be quietly settled by whoever picked the default threshold.

5.5 Wider ethical considerations

Privacy

Six directly identifying columns were removed before any modelling. A fraud system does not need to know a customer's name to judge whether a payment is out of character, and holding that data increases the harm caused by any future breach.

Transparency and the right to an explanation

When a payment is blocked the customer deserves a reason. This is why the model is built on interpretable behavioural features and why PCA was rejected for the final model. The LIME output in section 3.5 is the kind of explanation a call-centre agent could actually read aloud.

Human oversight

The system is designed to flag transactions for review, not to decide them. High-value blocks in particular should require human confirmation, and customers must have a route to challenge a decision and have it overturned.

Feedback loops

A quieter danger: if the model flags one group more often, that group generates more investigations, which generates more confirmed fraud records for that group, which teaches the next model to flag them even more. Monitoring must therefore track fairness metrics over time, not only accuracy.

Step 6 - Final Presentation and Communication

Two slide decks accompany this report, written for two audiences who need very different things.

Deck	Audience	Focus
Technical deck	Fellow data scientists and engineers	Method, metrics, model comparison, SHAP, the fairness audit, limitations
Business deck	Executives and risk owners	The problem in plain language, money saved, risks, and what to decide next

Files:

- Allen_Mendiola_Credit_Card_Fraud_Detection_Technical.pptx
- Allen_Mendiola_Credit_Card_Fraud_Detection_Business.pptx

The two decks deliberately do not share slides. The technical audience needs to know that PR-AUC was chosen over ROC-AUC because of class imbalance. The executive audience needs to know that the system pays for itself and carries a measured fairness risk. Presenting either group with the other's material is the most common way good analysis fails to land.

Step 7 - GitHub Repository

All code, results and documents for this project are published in a public repository, structured the way an open source project would be.

Repository: <https://github.com/alnmendiolaetc/Pillar5-Capstone>

Visibility: Public

7.1 Repository structure

Path	What it holds
src/	The eleven numbered scripts, run in order from 01 to 11
notebooks/	A walkthrough notebook that loads the saved results and shows the main findings, with outputs already executed
data/	Folder structure for the raw and processed data (contents not committed)
models/	The saved trained models: best_model.joblib, scaler.joblib, isolation_forest.joblib
reports/figures/	All 33 charts used in this report and in the slide decks
reports/tables/	Every metrics table and results file the documents are built from
instructions/	The original assignment brief, kept for reference
README.md	Setup, how to run the pipeline, and a summary of the findings
requirements.txt	Pinned library versions for reproducing the environment

7.2 What is deliberately not committed

Two things are excluded by .gitignore, and both exclusions are intentional.

- The virtual environment (venv/). It is large, machine-specific and completely rebuildable from requirements.txt.
- The data files. The raw dataset is 258 MB and the processed training file is 233 MB. GitHub rejects any single file above 100 MB, and committing data to version control is poor practice in any case.

This does not harm reproducibility. Running `src/01_download_data.py` fetches the dataset again from its public source, and every later script regenerates its own outputs from there. The empty data folders are kept in the repository with .gitkeep files so the structure is still visible.

7.3 Reproducing the work from the repository

```
git clone https://github.com/alnmendiolaetc/Pillar5-Capstone.git
cd Pillar5-Capstone
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python src/01_download_data.py
... then scripts 02 through 11 in order
```

Conclusion and Recommendations

What was achieved

- A fraud detection model was built achieving a PR-AUC of 0.8605, roughly 158 times better than chance, catching 88.1% of fraudulent transactions at the chosen operating point.
- The alert threshold was chosen on financial grounds rather than convention, producing an estimated net benefit of \$554,950 over the test period.
- Unsupervised clustering independently located a group with a 17.5% fraud rate without using any labels, and DBSCAN's noise points proved 51 times more likely to be fraudulent than grouped transactions.
- A full fairness audit across gender, age and city size demonstrated that withholding protected attributes did not by itself produce equal treatment.
- A mitigation was implemented and its cost measured honestly, rather than being recommended in the abstract.

Recommendations

Deploy, but with conditions

The model is good enough to be useful, at the 0.50 threshold, provided that alerts go to human reviewers rather than to automatic blocking, and that fairness metrics are monitored alongside accuracy from day one.

Decide the fairness question explicitly

Section 5.4 shows the cost of closing the protection gap. That is a decision for the business and its regulator, and it should be taken deliberately and recorded - not left to whoever configured the default threshold.

Retrain on a schedule

Fraud adapts. A model trained on last year's behaviour degrades. Monthly retraining with drift monitoring should be assumed as an operating cost, not treated as a project.

Validate on real data before trusting any of it

Every number in this report comes from simulated transactions. The methodology transfers; the specific figures do not. Real-world performance will be lower.

Limitations and Future Work

Limitations

- The data is simulated, so real fraud would be messier, more adaptive and harder.
- The source file was truncated at Excel's row limit, losing roughly 248,000 transactions and shortening the period covered.
- City population is a weak proxy for socioeconomic status; no income data exists in the dataset.
- The dataset contains no race or ethnicity field, so that dimension of the fairness audit could not be carried out at all.
- The cost model uses a flat \$5 review cost. A real bank would have tiered costs and would also price reputational damage from wrongly blocking a customer.
- Clustering and SHAP were computed on samples rather than the full dataset, for compute reasons.

Future work

- Model each card as a SEQUENCE of transactions and apply an LSTM or Transformer. This reframes the problem in a way that genuinely suits deep learning, unlike the flat-table formulation used here.
- Add velocity features - how many payments this card made in the last hour, day and week. These are among the strongest signals in production fraud systems and are absent here.
- Test adversarial robustness by simulating a fraudster who knows the model exists and deliberately keeps payments small and during daylight hours.
- Implement reweighting during training and compare it against the per-group thresholds used in section 5.4.
- Build a live scoring service and shadow-run it against the existing rules engine before allowing it to influence any real decision.

References

- Harris, S. (2020) Sparkov Data Generation. Simulated credit card transaction generator. Available at: https://github.com/namebrandon/Sparkov_Data_Generation
- Dataset mirror: Hugging Face, santosh3110/credit_card_fraud_transactions. Available at: https://huggingface.co/datasets/santosh3110/credit_card_fraud_transactions
- Lundberg, S. and Lee, S. (2017) 'A Unified Approach to Interpreting Model Predictions', Advances in Neural Information Processing Systems 30.
- Ribeiro, M., Singh, S. and Guestrin, C. (2016) '"Why Should I Trust You?": Explaining the Predictions of Any Classifier', KDD '16.
- Hardt, M., Price, E. and Srebro, N. (2016) 'Equality of Opportunity in Supervised Learning', Advances in Neural Information Processing Systems 29.
- Barocas, S., Hardt, M. and Narayanan, A. (2019) Fairness and Machine Learning: Limitations and Opportunities. Available at: <https://fairmlbook.org>
- Chawla, N. et al. (2002) 'SMOTE: Synthetic Minority Over-sampling Technique', Journal of Artificial Intelligence Research, 16, pp. 321-357.
- Chen, T. and Guestrin, C. (2016) 'XGBoost: A Scalable Tree Boosting System', KDD '16.
- Pedregosa, F. et al. (2011) 'Scikit-learn: Machine Learning in Python', Journal of Machine Learning Research, 12, pp. 2825-2830.
- Saito, T. and Rehmsmeier, M. (2015) 'The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets', PLOS ONE.
- US Equal Employment Opportunity Commission (1978) Uniform Guidelines on Employee Selection Procedures - the four-fifths (80%) rule.

Appendix A - How to Reproduce This Work

Setup

Windows, Python 3.10:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

Running the pipeline

The scripts are numbered and must be run in order. Each one writes files that the next one reads.

Script	What it does
01_download_data.py	Downloads the dataset from Hugging Face
02_data_overview.py	Data dictionary and quality checks
03_clean_and_engineer.py	Cleaning, feature engineering, train/test split
04_eda_figures.py	The nine EDA charts
05_train_models.py	Trains seven models, tunes the threshold
06_clustering.py	K-Means, DBSCAN, hierarchical clustering
07_explainability.py	SHAP, LIME, feature selection, PCA
08_bias_audit.py	The fairness audit and mitigation
09_build_report.py	Builds this document
10_build_pdf.py	Converts it to PDF
11_build_slides.py	Builds both slide decks

Folder layout

```
data/raw/ - the downloaded dataset
data/processed/ - cleaned train and test files
src/ - the eleven scripts above
models/ - saved trained models
reports/figures/ - every chart used in this report
reports/tables/ - every table and results file
notebooks/ - an executed walkthrough of the main findings
```

Environment

Python: 3.10.0

Operating system: Windows 11, CPU only

Random seed: 42 everywhere

Key libraries: pandas 2.3.3, scikit-learn 1.7.2, xgboost 3.2.0, shap 0.49.1